



國立台灣科技大學

資訊工程系

碩士學位論文

具情節記憶之高效輕量視覺語言模型應用於資源受限邊緣裝置部署

Efficient Tiny Vision-Language Models with
Episodic Memory for Resource-Constrained Edge
Deployment

研 究 生：Euhid Aman

學 號：M11315803

指導教授：鮑興國 博士

中 華 民 國 一 一 四 年 七 月 一 日



碩士學位論文指導教授推薦書

Master's Thesis Recommendation Form



M11015064

系所：資訊工程系
Department/Graduate Institute Department of Computer Science and Information Engineering

姓名：何昆霖
Name HO, KUN-LIN

論文題目：調適大型預訓練語言模型搭配新穎符號化方法於日誌異常偵測
(Thesis Title) Adapt Large Pre-trained Language Models to Log Anomaly Detection with a Novel Tokenization

係由本人指導撰述，同意提付審查。

This is to certify that the thesis submitted by the student named above, has been written under my supervision. I hereby approve this thesis to be applied for examination.

指導教授簽章：

Advisor's Signature

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

日期：

Date(yyyy/mm/dd)

2024, 7, 29



碩士學位考試委員審定書
Qualification Form by Master's Degree Examination Committee



M11015064

系所： 資訊工程系
Department/Graduate Institute Department of Computer Science and Information Engineering

姓名： 何昆霖
Name HO, KUN-LIN

論文題目： 調適大型預訓練語言模型搭配新穎符號化方法於日誌異常偵測
(Thesis Title) Adapt Large Pre-trained Language Models to Log Anomaly Detection with a Novel Tokenization

經本委員會審定通過，特此證明。

This is to certify that the thesis submitted by the student named above, is qualified and approved by the Examination Committee.

學位考試委員會

Degree Examination Committee

委員簽章：

Member's Signatures

王紹睿 邱永瑞

王紹睿 邱永瑞

指導教授簽章：

Advisor's Signature

王紹睿

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

王紹睿

系所（學程）主任（所長）簽章：

Department/Study Program/Graduate Institute Chair's Signature

日期：

Date(yyyy/mm/dd)

2024 / 7 / 29

摘要

大型視覺語言模型（Vision-Language Models, VLMs）在多模態任務上表現卓越，但高度依賴雲端基礎設施與高效能 GPU，難以直接部署於資源受限的邊緣環境。本論文針對記憶體低於 800 MB、純 CPU 推理、延遲低於 10 毫秒及任務關鍵場景零幻覺容忍等嚴苛約束，開發適用於裝置端部署的輕量視覺語言模型。

本論文提出兩套系統。**BitMar** 為 1400 萬參數、採用 1.58 位元三元量化的多模態模型，整合跨模態融合管線與外部情節記憶矩陣（512 槽位，128 維）。啟用情節記憶後，推理吞吐量提升約 7.5 倍，能耗降低 79%，並在實體追蹤、組合推理及視覺問答任務上獲得 3 至 4 個百分點的準確率提升，驗證了極限量化、跨模態融合與情節記憶在同一架構中共存的可行性。**EmberVLM** 為本論文主要貢獻，總計 1.64 億參數（4000 萬可訓練），採用四階段課程式訓練策略。其情節記憶模組採用高斯距離定址與新穎性觸發動態寫入機制，每步推理僅增加 1.2 MB 記憶體佔用及不足 3% 的浮點運算量，可將機器人選擇任務之巨集 F1 分數從 0.035 提升至 0.297。其視覺缺失精煉器（VA Refiner）為免訓練的幻覺抑制機制，透過雙視角邏輯差異分析與解碼器中間層神經元激活監控，無需額外前向傳遞即可抑制視覺上無依據的輸出。

EmberVLM 在三項任務關鍵基準上取得最佳成績：Top-N 機器人選擇（巨集 F1 = 0.42）、緊急視覺辨識（準確率 0.64）及地理災害理解（準確率 0.72）。完整推理堆疊在純 CPU 裝置上僅需 5.22 毫秒，記憶體佔用低於 800 MB。VA 精煉器將對抗性幻覺率從 29% 降至 22%。全訓練週期碳排放實測為 25.3 kg CO₂eq，展示了任務關鍵邊緣人工智慧的永續訓練路徑。

關鍵字— 邊緣人工智慧、視覺語言模型、情節記憶、幻覺抑制、模型量化、資源受限部署

Abstract

Large vision-language models (VLMs) achieve strong multimodal performance but depend entirely on cloud infrastructure and high-end GPU hardware, making them unsuitable for resource-constrained, offline, and privacy-sensitive deployment scenarios. This thesis develops compact VLMs for on-device inference under simultaneous hard constraints: under 800 MB RAM, CPU-only execution, sub-10 ms latency, and zero tolerance for hallucinated outputs in mission-critical settings.

Two systems are presented. **BitMar** is a 14M-parameter, 1.58-bit ternary-quantized multimodal model integrating a cross-modal fusion pipeline with an external episodic memory matrix (512 slots, 128 dimensions). Enabling episodic memory yields approximately $7.5\times$ throughput improvement and 79% energy reduction over the memory-free baseline, alongside 3–4 percentage-point accuracy gains on entity tracking, compositional reasoning, and visual question answering. BitMar establishes the foundational compatibility of extreme quantization, cross-modal fusion, and episodic memory in a single architecture. **EmberVLM** is the primary contribution: a 164M-parameter VLM (40M trainable) with a four-stage training curriculum. Its Episodic Memory Module employs Gaussian distance-based addressing and novelty-triggered online writes, adding only 1.2 MB RAM and under 3% extra FLOPs per inference step, boosting robot-selection Macro-F1 from 0.035 to 0.297. Its Visual Absence (VA) Refiner is a training-free hallucination control mechanism fusing logit-space dual-view consistency with neuron-level mid-decoder activation monitoring, requiring no additional forward passes.

EmberVLM achieves the best reported results among sub-billion-parameter models on three mission-critical benchmarks: Top-N Robot Selection (Macro-F1 = 0.42), emergency visual recognition (Accuracy = 0.64), and geospatial disaster understanding (Accuracy = 0.72). The full inference stack runs at 5.22 ms per sample on a CPU-only device with under 800 MB RAM. The VA Refiner reduces adversarial hallucination rate from 29% to 22%. Total measured training emissions are 25.3 kg CO₂eq, demonstrating that

mission-critical multimodal AI can be trained sustainably.

Keywords— Edge AI, Vision-Language Models, Episodic Memory, Hallucination Mitigation, Model Quantization, Resource-Constrained Deployment

Acknowledgements

This thesis would not exist without the people who made the past two years possible.

I am deeply grateful to my advisor, Prof. Hsing-Kuo Pao, who supported this research from the very first conversation about the idea all the way through to the final results. He never dismissed what might have seemed like insane directions, and that trust made all the difference. I am equally grateful to Prof. Giovanni Beltrame, whose guidance during my time in Montreal shaped my thinking in ways that run through every chapter of this work. I genuinely thank them both for believing that the research was not crazy.

Ghaluh Indah Permata Sari has been a wonderful friend and a constant presence in the lab. She brought a cheerfulness and warmth to every day that made the long hours easier, and I am fortunate to have worked alongside her.

Rodrigo Garcia has been a great friend and, in many ways, an elder brother throughout this journey. He and his fiancée Alexa listened through every difficulty, every frustration, and every late-night conversation with patience and genuine care. They understood what these two years really involved, and that understanding meant more than I can say. Roy listened across a twelve-hour time difference and never gave up, and I am grateful for that constancy. The time spent with all of them passed far too quickly.

Esteban Carlin and I spent countless hours discussing science, going down research directions that seemed impractical and sometimes even absurd. The best ideas came out of those conversations, and BitMar is one of

them.

Alex and Luis always asked the hard questions, the kind that force clarity and honesty about what the work actually is. Those questions made the research better.

To all the members of the MIST Lab in Montreal: thank you. I had the privilege of attending conferences and events in Toronto, Montreal, Surabaya, Bali, and Shanghai over the course of this degree. I could not have imagined when I started that the research would take me to all those places, and the people I met along the way made each of them memorable.

My deepest gratitude belongs to my parents, my Mummy and Papa. They believed in me when it was not easy to do so. They spoke with me, understood my problems, and remained my constant support across the distance. Being an only son carries its own weight, and I know what they went through while I was away. I hope this work is something they can be proud of.

Finally, a sincere and heartfelt thanks to Ruchismita Das, for being present through the last three to four years, through the good moments and the difficult ones alike. I am sorry for the trouble that came with it. Thank you for still believing in me, and for understanding.

Contents

Recommendation Form	I
Qualification Form	II
Abstract in Chinese	III
Abstract in English	IV
Acknowledgements	VI
Contents	VIII
List of Figures	XIII
List of Tables	XVIII
List of Algorithms	XX
1 Introduction	1
1.1 Background and Motivation	1
1.2 Research Challenges	3
1.2.1 Memory and Compute Constraints	3
1.2.2 Hallucination at the Edge	4
1.2.3 Knowledge Staleness and Distributional Shift	5
1.3 Research Overview and Contributions	6
1.4 Thesis Organization	8
2 Related Work	10
2.1 Tiny and Edge-Deployable VLMs	10

2.2	Model Quantization for Efficient Inference	12
2.3	Episodic Memory in Neural Models	13
2.4	Hallucination Detection and Control in VLMs	15
2.5	Mission-Critical Visual Recognition Benchmarks	16
2.6	Low-Carbon and Sustainable AI	18
3	BitMar: Low-Bit Multimodal Foundation for Edge Deployment	19
3.1	Motivation and Research Goals	19
3.2	BitMar Architecture Overview	20
3.3	Text Encoder	21
3.3.1	Architecture	21
3.3.2	Quantization	23
3.3.3	Attention Sinks for Long-Context Processing	23
3.4	Vision Encoder	24
3.5	Cross-Modal Fusion	24
3.6	Episodic Memory Module	25
3.6.1	Memory Matrix Specification	25
3.6.2	Writing Mechanism	25
3.6.3	Reading Mechanism	25
3.6.4	Regularization and Forgetting	26
3.7	Decoder with Attention Sinks	26
3.8	Training Objectives and Adaptive Curriculum	27
3.8.1	Multi-Component Objective	27
3.8.2	Adaptive Training Controller	27
3.9	Dataset and Training Configuration	27
3.10	Quantization Effectiveness Analysis	28
3.11	Experimental Results	30

3.11.1	Language Understanding Benchmarks	30
3.11.2	BabyLM Evaluation Suite	32
3.11.3	Episodic Memory Ablation	33
3.12	Implementation Details	34
3.12.1	Hyperparameters	34
3.12.2	Training Corpus	36
3.12.3	Tokenizer and Preprocessing	36
3.12.4	External Memory Offloading	38
3.13	Lessons Learned and Research Directions	39
4	EmberVLM: Efficient Edge Vision-Language Model	41
4.1	Overview and Design Philosophy	41
4.2	Base Multimodal Pipeline	42
4.2.1	Vision Encoder	42
4.2.2	Language Backbone	42
4.2.3	Parameter Breakdown	44
4.3	Stable Gated Fusion	45
4.4	Task-Specific Adaptation: Top- K Robot Selection Head	46
4.5	Episodic Memory Module	47
4.5.1	Overview and Distinction from BitMar	47
4.5.2	Memory Matrix Specification	47
4.5.3	Scope Detector	48
4.5.4	Memory Addressing and Retrieval	48
4.5.5	Novelty Detection and Dynamic Writes	49
4.5.6	Stage 5 Knowledge Consolidation	49
4.5.7	Complete Memory Inference Procedure	50
4.5.8	Memory Operating Modes	50

4.6	Visual Absence (VA) Refiner	50
4.6.1	Motivation	50
4.6.2	Dual-View Consistency Pathway	52
4.6.3	Neuron-Level Activation Pathway	53
4.6.4	Combined Visual Absence Score	53
4.6.5	Type-Aware Intervention	54
4.6.6	Temporal Burst Detection	54
4.6.7	Mission Uncertainty Calibration	55
4.6.8	Complete VA Refiner Inference Procedure	55
4.7	Four-Stage Training Curriculum	55
4.7.1	Stage 1: Visual-Language Alignment	56
4.7.2	Stage 2: Multimodal Instruction Tuning	57
4.7.3	Stage 3: Robot Fleet Selection	59
4.7.4	Stage 4: Latent Chain-of-Thought Integration	59
4.8	Dynamic Tiered Memory Management	61
4.9	Carbon Footprint Analysis	62
5	Experiments and Results	64
5.1	Experimental Setup	64
5.1.1	Hardware and Software	64
5.1.2	Evaluation Protocol	64
5.1.3	Baseline Models	64
5.1.4	Robot Selection Dataset	65
5.2	Mission-Critical Benchmark Results	66
5.2.1	Top-N Robot Selection	66
5.2.2	VERI-Emergency	68
5.2.3	GEO-Bench-VLM	69

5.3	General Multimodal Benchmarks	70
5.4	Episodic Memory Ablation	72
5.5	Visual Absence Refiner Ablation	73
5.6	Edge Deployment Latency Analysis	75
5.7	Mechanistic Analysis of Hallucinations	76
6	Discussion	77
6.1	Research Progression: From BitMar to EmberVLM	77
6.2	The Case for Episodic Memory at the Edge	79
6.3	Hallucination as a Systems Problem	80
6.4	Cross-Architecture Transfer of the VA Refiner	81
6.5	Sustainable Edge AI	82
6.6	Limitations	82
7	Conclusions and Future Work	84
7.1	Conclusions	84
7.2	Future Work	87
	References	89

List of Figures

3.1	BitMar Architecture. The model processes multimodal inputs: text tokens and DiNOv2-compressed image features. Quantized encoders (1.58-bit) generate compact text and vision embeddings (z, v) , which are fused via cross-modal attention into shared query representations. A sliding-window attention mechanism enables long-context processing. A fixed episodic memory matrix $(K \times C)$ stores and retrieves multimodal context vectors through quantized read/write weights, supporting optional SD-card offloading for edge deployment.	22
3.2	Quantization effectiveness over training. E_q (fraction of ternary weights set to zero) stabilizes at 42.8%, demonstrating efficient compression without downstream performance degradation.	30
3.3	Episodic memory slot activation patterns across training checkpoints. Each row corresponds to a different checkpoint, ranging from early training (top, Epoch 0, step 20k) to late training (bottom, Epoch 9, step 97k). Early checkpoints show scattered, weak activations with minimal slot specialization. By the final checkpoint, memory slots develop strongly differentiated activation profiles, with specific slots specializing for distinct semantic categories. This convergence confirms that the episodic memory organizes stored representations by semantic content rather than recency.	31

4.1	EmberVLM architecture overview. The system integrates a frozen DINOv2-Small vision encoder, a SmolLM-135M language backbone, a Stable Gated Fusion module, an Episodic Memory Module, and the Visual Absence (VA) Refiner. Solid arrows denote forward-pass data flow; the memory matrix is outside the gradient graph and is updated algebraically at deployment time.	43
4.2	EmberVLM parameter distribution. DINOv2-Small (21.1M) and the frozen layers of SmolLM-135M account for the majority of total parameters and are never updated. The 40M trainable parameters are concentrated in the Fusion Module, bottleneck adapter, and Reasoning Head.	44
4.3	Neuron activation heatmaps. Top: For hallucinated cases, activation patterns are nearly identical with and without the image, indicating generation from language priors. Bottom: For correctly grounded cases, activations change substantially when the image is removed, demonstrating reliance on actual visual content.	54
4.4	EmberVLM four-stage training curriculum. Stage 1 freezes both backbones and trains only the Fusion Module for visual-language alignment. Stage 2 unfreezes the language backbone for instruction tuning. Stage 3 re-freezes the backbone and trains the Reasoning Head for robot fleet selection. Stage 4 trains the Reasoning Head via InfoNCE contrastive loss to align internal step-embeddings with procedurally generated chain-of-thought rationales.	57

4.5	Representative training samples across stages. Stage 1 alignment samples pair images with short captions. Stage 2 instruction tuning samples involve multi-turn visual dialogue. Stages 3–4 use structured robot-selection prompts with multi-label annotations and programmatic chain-of-thought rationales.	59
4.6	Text-to-patch attention progression during Stage 1 alignment training. Each row shows a training checkpoint (early, mid, late). The left column shows the input image with its associated caption; the right column shows the corresponding text-to-patch attention heatmap. Warm colors (red/yellow) indicate high attention weight on image patches. As training progresses, diffuse initial attention concentrates onto the semantically relevant regions of the scene, demonstrating the Fusion Module’s acquisition of cross-modal visual grounding.	60
4.7	Target edge deployment environments. EmberVLM is designed for CPU-only hardware platforms such as the AMD Ryzen 3 4100 (3.80 GHz, 32 GB RAM, no discrete GPU), representative of resource-constrained field units used in emergency response, autonomous robotics, and remote sensing applications.	62
5.1	Robot Selection dataset statistics. (a) Per-class label counts across train/val/test splits. Underwater Robot and Robot with Wheels are the most scarce classes, contributing to high prediction variance on the validation split. (b) Class co-occurrence matrix showing which robot types are frequently annotated together in the same scenario.	66

5.2	Robot Selection input and output statistics. (a) Input scenario descriptions are predominantly 20–80 tokens long, well within the 256-token maximum sequence length used in Stages 3–4. (b) The single-label output class distribution confirms that multi-robot scenarios are common, validating the multi-label evaluation protocol.	66
5.3	Distribution of required robot types per scenario. Most scenarios require two or more robot types, confirming that single-label classification is insufficient and validating the multi-label evaluation protocol using Macro-F1 and Jaccard similarity.	67
5.4	Five-axis normalized benchmark radar. EmberVLM (red, filled) leads on MC Score (≈ 0.95 , collapsing Top-N Robot Selection, VERI-Emergency, and GEO-Bench) and dominates the Latency Efficiency axis (inverted: $\text{min_latency}/\text{latency}$, so CPU-only EmberVLM = 1.0 and GPU baselines near zero). InternVL3-1B sweeps the three general-benchmark axes but scores near zero on latency efficiency, confirming it is unsuitable for the target hardware despite its higher VQA accuracy. The MC Score and Latency Efficiency axes directly govern edge deployment viability.	71
5.5	Per-class F1 radar chart across memory modes (Top-N Robot Selection). READ_ONLY achieves the broadest coverage overall. ON-LINE_UPDATE shows stronger per-class recall on the Underwater Robot axis, the scarcest class (13 validation samples), suggesting live writes partially address rare-morphology prediction.	73

5.6 VA Refiner hallucination drop on POPE Adversarial split. Each bar shows the hallucination rate before and after VA Refiner activation. EmberVLM achieves the largest absolute reduction (-7pp , from 29% to 22%), while SmolVLM2-256M also shows a substantial improvement (-3.47pp). InternVL3-1B and SmolVLM-256M show no benefit, consistent with their strong internal visual grounding. . . . 74

List of Tables

3.1	BitMar training configuration. Key hyperparameters used during the 10-epoch training run on the 100M-token corpus.	29
3.2	Language understanding benchmark performance. ✓ = natively 1-bit trained. Scores are accuracy (%). WG = WinoGrande, CQA = CommonsenseQA, ARC-E = ARC-Easy.	30
3.3	BitMar BabyLM evaluation. Average finetuned NLP accuracy: 60.5%.	32
3.4	Inference efficiency with and without episodic memory.	33
3.5	Zero-shot accuracy deltas (Δ pp) with episodic memory enabled.	33
3.6	BitMar architectural hyperparameters. Full specification of the 14M-parameter ternary-quantized architecture.	34
3.7	BitMar training hyperparameters. Optimizer, schedule, memory, and hardware settings used across the full 10-epoch run.	37
3.8	BitMar training corpus composition. The 101M-token corpus combines 50M multimodal tokens and 50M text-only tokens, with a 1M-token stratified hold-out set.	38
4.1	EmberVLM parameter breakdown. Total: 164M parameters, 40M trainable.	44
4.2	EmberVLM architectural specification. Component-level summary of the 164M-parameter system, including encoder, backbone, memory, and VA Refiner configuration.	45
4.3	EmberVLM episodic memory operating modes. The three runtime modes control whether the memory matrix is read, written, or inactive during inference.	52
4.4	EmberVLM training datasets across all stages.	58

4.5	EmberVLM training hyperparameters by stage.	58
4.6	Training carbon emissions (CodeCarbon-measured). All values are measured, not estimated. Training on $2 \times A100$ GPUs.	63
5.1	Performance on mission-critical visual recognition benchmarks. Time(s) = total wall-clock over the full evaluation set on the AMD Ryzen 3 4100 CPU. OverRx / UnderRx = over/under-reaction rates (lower UnderRx is safer).	67
5.2	Robot fleet selection ablation. 103-sample validation split. ECE: lower is better ($\downarrow$); all others: higher is better ($\uparrow$).	68
5.3	Top-N Robot Selection dataset split statistics. Per-class sample counts across train, validation, and test splits. Classes are imbalanced, with Underwater Robot and Robot with Wheels being the least frequent. . .	69
5.4	General multimodal benchmarks and inference efficiency.	70
5.5	Memory mode ablation across mission-critical benchmarks.	72
5.6	VA Refiner component ablation across all mission-critical benchmarks. Memory mode = ONLINE_UPDATE for all rows. $\downarrow$ = lower is better.	73
5.7	POPE hallucination rates by split. Evaluated on the complete 9,000-sample POPE dataset. $\Delta \downarrow$ = absolute hallucination rate reduction; lower is better.	74
5.8	CPU-only edge inference latency on AMD Ryzen 3 4100. Full stack = Model + VA Refiner + Memory.	75
6.1	Architectural progression from BitMar to EmberVLM.	78

List of Algorithms

1	BitMar episodic memory read/write procedure.	26
2	Adaptive Training Controller (ATC) for modality collapse prevention.	28
3	EmberVLM episodic memory: scope detection, retrieval, novelty write, and LRU-LFU eviction.	51
4	Visual Absence (VA) Refiner: per-token hallucination detection and logit intervention.	56

Chapter 1 Introduction

1.1 Background and Motivation

Vision-Language Models (VLMs) represent one of the most transformative advances in modern artificial intelligence, enabling systems to jointly reason over visual and textual information for tasks including image captioning, visual question answering, scene understanding, and multimodal dialogue. Landmark architectures such as Flamingo [1], BLIP-2 [2], GPT-4V [3], and Gemini [4] have demonstrated that scaling model parameters alongside multimodal pretraining yields qualitative improvements in cross-modal grounding and generation. This scaling trend has continued unabated, with frontier VLMs now exceeding hundreds of billions of parameters.

However, this impressive progress carries a critical structural dependency: virtually all state-of-the-art VLMs are designed for and evaluated in *cloud-hosted* settings. They assume access to GPU clusters, high-bandwidth internet connectivity, centralized storage, and milliseconds-to-seconds latency budgets that are acceptable only when a remote server handles inference. This assumption is so deeply embedded in the design of modern VLMs that it is rarely articulated as a limitation, yet it categorically excludes an important and growing class of real-world deployment scenarios.

Consider the following concrete deployment contexts:

- **Disaster response robotics.** Following earthquakes, floods, or structural collapses, autonomous robots must navigate rubble, classify environments, and select appropriate robotic assets for deployment.

Network infrastructure is typically destroyed, and round-trip latency to a cloud server is unacceptable when a robot is making split-second decisions in a life-critical zone.

- **Aerial surveillance drones.** Unmanned aerial vehicles conducting reconnaissance or border monitoring in remote terrain operate beyond cellular or satellite connectivity windows. They must perform real-time scene classification and anomaly detection entirely on-device.
- **Autonomous vehicles.** Road safety systems require sub-50ms visual reasoning to detect obstacles, pedestrians, and road conditions. Transmitting raw sensor data to a cloud server and waiting for a response is physically impossible at highway speeds.
- **Industrial inspection.** Facilities handling hazardous materials or electromagnetic-sensitive equipment frequently operate in radio-frequency-silent zones. Visual inspection AI must run locally on embedded hardware.
- **Medical edge devices.** Point-of-care diagnostic devices in low-resource clinical settings lack reliable internet and must perform visual analysis (e.g., dermatological screening, wound assessment) on battery-powered hardware.

All of these scenarios impose hard, simultaneous constraints: limited VRAM or SRAM (often under 1 GB), CPU-only inference (no discrete GPU), zero-tolerance for hallucinated outputs in safety-critical decisions, and complete independence from network connectivity. These constraints are not engineering inconveniences to be addressed incrementally; they are categorical requirements that current VLMs cannot meet.

The central challenge of this thesis is therefore not merely “making models smaller.” Smaller models have existed for some time (SmolVLM-256M, MobileVLM, LLaVA-Phi), yet none are designed for simultaneous CPU-only inference, sub-10ms latency, under 800 MB RAM, and zero-hallucination tolerance in mission-critical classification tasks. The research gap is this: **achieving reliable, hallucination-suppressed, contextually aware multimodal reasoning under simultaneous constraints of parameter budget, memory footprint, computational budget, and connectivity independence.**

1.2 Research Challenges

This thesis identifies three primary, interconnected challenges that any viable edge VLM must address.

1.2.1 Memory and Compute Constraints

Even sub-billion-parameter VLMs impose memory and latency requirements that exceed the capabilities of commodity edge hardware. SmolVLM-256M, one of the smallest publicly available VLMs, requires a discrete GPU for practical inference and achieves 941 ms per sample even in that configuration. InternVL3-1B requires 1,978 MB VRAM and achieves 1,566 ms per sample on the target CPU-only deployment hardware used in this thesis (AMD Ryzen 3 4100). These numbers are not competitive for real-time edge deployment where latency budgets are in the tens of milliseconds and no discrete GPU is available.

The fundamental technical mismatch arises from the full-precision floating-point arithmetic and large activation tensors that current VLMs require, combined with transformer attention complexity that scales quadratically with sequence length. Even architectures specifically marketed as “efficient” have not been designed for the specific combination of CPU-only inference, sub-800 MB RAM, and sub-10ms latency that defines practical edge deployment.

1.2.2 Hallucination in Mission-Critical Settings

Hallucination, the generation of tokens unsupported by visual evidence in the input image, is a fundamental and well-documented failure mode in all current VLMs. A model may confidently describe an object that is absent from the image, or assign incorrect attributes to present objects, purely by relying on statistical language priors from pretraining.

In cloud-based consumer applications, hallucination is a quality issue. In mission-critical edge settings, it is a safety failure. If a robot selection system hallucinates the presence of a “Drone” when the scene actually shows a flooded corridor requiring an “Underwater Robot,” the wrong asset is deployed and the mission fails. If an emergency recognition system hallucinates that a burning structure is a “normal building,” emergency response resources are not dispatched.

Existing hallucination mitigation strategies are largely infeasible at the edge. Methods based on grounding supervision or retraining require annotated data and compute that are not available during deployment. Visual Contrastive Decoding (VCD) [5], while effective, requires an extra for-

ward pass with perturbed visual input, doubling inference cost, which is unacceptable at the latency budgets required. What is needed is a training-free, zero-extra-forward-pass mechanism that operates within the model’s existing inference graph.

1.2.3 Knowledge Staleness and Distributional Shift

Edge VLMs are typically frozen after deployment: their parametric weights do not change in the field. Yet the environments they encounter are dynamic and may differ substantially from the training distribution. Novel robot morphologies, previously unseen disaster scenarios, or unfamiliar terrain types may appear after deployment.

Without a mechanism for incorporating new contextual knowledge, models collapse under distributional shift. This is starkly demonstrated by the EmberVLM NoMemory baseline evaluated in this thesis: without episodic memory conditioning, the model degrades into a near single-class predictor on the robot selection task, achieving validation accuracy of 0.097 and macro-F1 of 0.035. It over-indexes on the most frequent class while entirely failing on minority classes. The episodic memory mechanism must therefore be more than a retrieval cache: it must provide a principled way to accumulate new deployment-time knowledge without catastrophic forgetting, without retraining, and without violating the memory budget of the target hardware.

1.3 Research Overview and Contributions

This thesis presents a unified research trajectory toward addressing these three challenges simultaneously, structured around two complementary research contributions.

BitMar. BitMar [6] is the first published contribution of this thesis research. BitMar is a 14M-parameter, 1.58-bit ternary-quantized multimodal model that establishes the foundational proof-of-concept for the thesis. It demonstrates that (i) aggressive ternary quantization ($-1, 0, +1$ weights at 1.58 bits/parameter) is compatible with cross-modal multimodal fusion, (ii) an external episodic memory module provides measurable efficiency and accuracy benefits in the quantized multimodal setting, and (iii) the combination of quantization, cross-modal fusion, and episodic memory can co-exist in a 14M-parameter architecture. BitMar validated the core thesis proposition before the larger EmberVLM system was built.

EmberVLM. EmberVLM is the primary architectural contribution of this thesis. It is a 164M total parameter (40M trainable) tiny VLM purpose-built for mission-critical edge deployment. EmberVLM introduces two novel modules: an Episodic Memory Module with scope detection, Gaussian-addressed retrieval, novelty-triggered online writes, and knowledge consolidation distillation; and a Visual Absence (VA) Refiner combining logit-space dual-view consistency with neuron-level mid-decoder activation monitoring. EmberVLM achieves the best reported performance among sub-billion-parameter models on three mission-critical benchmarks, at 5.22 ms per sample on CPU-only hardware with under 800 MB RAM.

The specific contributions of this thesis are:

1. **BitMar**: The first architecture to jointly integrate 1.58-bit multimodal encoding with an external episodic memory for edge deployment. BitMar achieves approximately $7.5\times$ throughput improvement and 79% energy reduction versus the memory-free baseline, delivering 3–4 percentage-point gains on entity tracking, compositional reasoning, and visual question answering at 14M parameters.
2. **EmberVLM architecture**: A tiny VLM (164M total parameters, 40M trainable) employing a four-stage training curriculum (visual-language alignment, multimodal instruction tuning, task adaptation, and latent chain-of-thought integration), achieving competitive mission-critical performance against models $5\text{--}10\times$ larger.
3. **Episodic Memory Module**: A dynamically loadable memory system with fixed overhead (1.2 MB RAM, $<3\%$ extra FLOPs per inference step), a lightweight scope detector, Gaussian distance-based addressing, novelty-triggered online writes with a hybrid LRU-LFU eviction policy, and an optional Stage 5 knowledge consolidation distillation phase. The module boosts robot-selection macro-F1 from 0.035 to 0.297.
4. **Visual Absence (VA) Refiner**: A training-free plug-in hallucination control mechanism fusing dual-view logit discrepancy with neuron-level mid-decoder activation monitoring (layers 10–14 of SmolLM-135M), applied via type-aware thresholds and temporal burst detection. The VA Refiner reduces POPE Adversarial hallucination rate from 29% to 22%, an absolute reduction of 7 percentage points.

5. **Comprehensive mission-critical evaluation:** State-of-the-art results for sub-billion-parameter models on Top-N Robot Selection (Macro-F1 = 0.42), VERI-Emergency (Accuracy = 0.64), and GEO-Bench-VLM (Accuracy = 0.72); inference at 5.22 ms per sample on CPU-only hardware with <800 MB RAM; total measured training emissions of 25.3 kg CO₂eq.

1.4 Thesis Organization

The remainder of this thesis is structured as follows.

Chapter 2 surveys related work across five areas: tiny and edge-deployable VLMs, model quantization, episodic memory in language and multimodal systems, hallucination detection and control, and mission-critical visual recognition benchmarks.

Chapter 3 presents BitMar in full detail, including its architecture, training objectives, dataset, and experimental results. This chapter also identifies the open questions that motivated the design of EmberVLM.

Chapter 4 presents the EmberVLM architecture, covering the base multimodal pipeline, the Episodic Memory Module, the Visual Absence Refiner, and the four-stage training curriculum. Chapter 5 presents experimental results for both BitMar and EmberVLM, including mission-critical benchmark evaluations, ablation studies, edge deployment latency, and mechanistic interpretability of hallucinations.

Chapter 6 synthesizes findings across both works, discussing the research progression from BitMar to EmberVLM, the empirical case for episodic memory at the edge, hallucination as a systems-level problem, and impli-

cations for sustainable AI development.

Chapter 7 concludes the thesis and outlines directions for future research.

Section 3.12 in Chapter 3 provides complete hyperparameter tables, corpus statistics, and preprocessing details for BitMar. Section 4.9 in Chapter 4 provides the full architectural specification and carbon tracking methodology for EmberVLM.

Chapter 2 Related Work

This chapter surveys prior work across five research areas directly relevant to the contributions of this thesis: tiny and edge-deployable vision-language models, model quantization for efficient inference, episodic memory in language and multimodal systems, hallucination detection and control in VLMs, and mission-critical visual recognition benchmarks. Each section contextualizes prior work relative to the thesis contributions and concludes with a statement of how this thesis advances beyond the existing literature.

2.1 Tiny and Edge-Deployable Vision-Language Models

The rapid scaling of VLMs has prompted a parallel line of work on compact architectures that preserve multimodal capability under reduced parameter budgets. SmolVLM [7] demonstrates that sub-billion-parameter models paired with strong vision encoders can achieve competitive results on standard benchmarks while being deployable on commodity hardware. The authors show that careful data curation and efficient vision tokenization significantly impact quality at small scales. However, even SmolVLM-256M requires a discrete GPU for practical inference and does not target CPU-only deployment.

MobileVLM [8] proposes lightweight vision-language connectors and mobile-optimized language backbones for responsive on-device inference. The architecture demonstrates that the vision-language bridge is a critical

bottleneck in compact VLMs, and that lightweight depthwise-separable designs can reduce parameter count. MobileVLM V2 [9] extends this with token compression and improved throughput. While these models represent genuine progress, they target GPU-based mobile devices rather than the CPU-only, RAM-limited deployments addressed in this thesis.

LLaVA-Phi [10] combines a compact Phi-2 language model with a CLIP vision encoder, showing that 2.7B-parameter instruction-tuned VLMs can approach much larger models at substantially lower inference cost. This is relevant to robotics and edge deployment, but 2.7B parameters still exceeds the practical limits of the target hardware by more than an order of magnitude. TinyGPT-V [11] further reduces parameters by employing efficient patch embedding strategies and distillation from larger teacher models. MiniCPM-V [12] pursues efficiency through selective parameter sharing and dynamic token pruning, achieving strong results in the 2–3B parameter range. H2OVL-Mississippi targets inference on low-resource hardware by combining aggressive pruning with knowledge distillation.

The consistent pattern across all prior compact VLM work is that none simultaneously targets CPU-only inference, under 800 MB RAM, sub-10ms per-sample latency, and zero-hallucination reliability for mission-critical classification. Most assume GPU acceleration; all treat hallucination control and epistemic memory as orthogonal concerns rather than core design requirements. This thesis addresses the full intersection of these requirements with a unified architecture.

2.2 Model Quantization for Efficient Inference

Quantization reduces model memory and compute requirements by representing weights and/or activations at lower bit-widths than standard 32-bit floating point. BitNet [13] demonstrates that transformer language models can be trained from scratch with 1-bit weights without catastrophic accuracy degradation, enabling highly memory-efficient inference on specialized hardware. The key insight is that signed integer multiply-accumulate operations can replace expensive floating-point operations throughout the forward pass.

BitNet b1.58 [14] extends this to ternary weights $\{-1, 0, +1\}$, which require only 1.58 bits per parameter (the binary entropy of a ternary distribution). At this precision, nearly all multiply operations can be replaced by additions and subtractions, dramatically reducing compute on hardware without dedicated low-precision units. BitNet b1.58 achieves near-lossless perplexity compared to full-precision baselines when trained at sufficient scale, validating ternary quantization as a practical compression strategy.

GPTQ [15] provides one-shot post-training quantization via approximate second-order information, enabling efficient 3–4 bit quantization of already-trained large models without backpropagation. This is complementary to BitNet-style native quantization and is widely used to compress pre-trained models for deployment. AWQ [16] proposes activation-aware weight quantization that identifies and protects salient channels most sensitive to quantization error, achieving strong 4-bit compression across language and multimodal models.

BitMar, the first contribution of this thesis, applies 1.58-bit ternary

quantization to a multimodal pipeline: both the text encoder and all downstream fusion, memory, and decoder projections operate under ternary weight constraints. This is a direct extension of BitNet b1.58’s quantization scheme to the cross-modal setting, and to the best of our knowledge is the first architecture to demonstrate that ternary quantization is compatible with cross-modal attention fusion and episodic memory conditioning simultaneously.

2.3 Episodic Memory in Language and Multimodal Models

The idea of augmenting neural networks with external, addressable memory has a long history. Differentiable Neural Computers (DNC) [17] introduced the first complete architecture for learning explicit read and write operations over an external memory matrix, demonstrating that neural networks can solve complex algorithmic tasks requiring long-range structured recall. While DNCs operate only on symbolic sequences, they established the key design principles, including content-based addressing, soft writes, and read-weight computation, that inform modern memory-augmented VLMs.

Memorizing Transformers [18] show that augmenting standard transformer language models with external memory lookups over past key-value pairs improves long-context modeling without changing core model weights. This work demonstrates that memory retrieval can be separated from parametric knowledge, allowing a frozen model to leverage growing context stores. Retrieval-Augmented Generation (RAG) [19] takes a related approach, connecting generative models to large external document stores to

improve factuality and reduce hallucination on knowledge-intensive tasks. However, RAG targets server-scale infrastructure: dense passage retrieval from millions of documents requires GPU-accelerated vector search engines that are impractical at the edge.

Larimar [20] presents a memory-controlled LLM framework in which episodes are written to and retrieved from a structured store using a differentiable interface, enabling the model to update its knowledge without full retraining. This is conceptually close to the EmberVLM episodic memory design, though Larimar targets large language models on GPU servers rather than tiny VLMs on CPU-only edge hardware.

EGO and related selective episodic memory strategies explore how memory writes can be conditioned on novelty signals to prevent the memory matrix from being overwritten with redundant common experiences. This informs EmberVLM’s novelty-triggered write mechanism, where a minimum-distance novelty score gates whether an incoming embedding warrants a new memory write.

A critical distinction between the memory systems in this thesis and server-scale RAG or database-backed retrieval: both BitMar and EmberVLM use *fixed-size, on-device episodic matrices* that fit entirely within the primary compute device’s RAM (1.2 MB). No external database query, network request, or dense retrieval index is involved. Memory reads and writes are pure matrix operations that complete in microseconds. This makes the memory system viable for real-time edge inference in a way that server-scale retrieval is not.

2.4 Hallucination Detection and Control in VLMs

Hallucination in VLMs, the generation of tokens unsupported by the input image, has been extensively documented and benchmarked. Rohrbach et al. [21] showed that image captioning models systematically generate statistically likely but visually absent objects, even in high-performing systems. They demonstrate that hallucination correlates with co-occurrence statistics in training data rather than actual visual evidence. This foundational finding motivates all subsequent hallucination mitigation research.

The POPE benchmark [22] formalizes object hallucination evaluation as a polling-based yes/no question protocol: given an image, models are asked whether specific objects are present. POPE includes adversarial splits where absent objects are specifically chosen to have high co-occurrence with genuinely present objects, making them maximally likely to be hallucinated. POPE has become the standard evaluation for object-level hallucination across the VLM literature.

Visual Contrastive Decoding (VCD) [5] addresses hallucination without retraining by contrasting output distributions from original and spatially distorted visual inputs. Tokens whose generation probability is not significantly affected by the visual content are penalized, encouraging the model to rely on actual image features rather than language priors. VCD is effective across multiple large VLM families, but it requires a complete second forward pass with perturbed input, doubling the inference cost. For the CPU-only, sub-10ms deployment targets of this thesis, this overhead is unacceptable.

Instruction-following calibration approaches train models to express

uncertainty when visual evidence is insufficient, requiring either additional labeled data or specialized fine-tuning pipelines. Grounding supervision methods add explicit visual localization objectives during training to reduce the gap between language-prior generation and visually evidenced generation. Both classes of methods require retraining or supplementary annotation, making them infeasible for deployment-time use on frozen edge models.

The Visual Absence (VA) Refiner introduced in this thesis occupies a previously unexplored position in the mitigation landscape: it is training-free, requires no extra forward passes, needs no labeled grounding data, and operates by monitoring the model’s own internal neuron-level activation patterns during a single standard forward pass. By hooking into mid-decoder FFN layers (10–14 of SmolLM-135M), the VA Refiner detects the characteristic activation signature of hallucinated tokens, specifically symmetric patterns that do not change when the visual input is removed, and applies type-aware logit penalties to suppress them. This mechanistic grounding distinguishes the VA Refiner from all prior approaches and makes it uniquely suitable for frozen edge deployment.

2.5 Mission-Critical Visual Recognition Benchmarks

Standard VLM benchmarks such as VQA v2 [23], GQA [24], and SugarCrepe [25] are designed to measure general visual question answering and compositional reasoning capabilities across diverse image domains. While valuable for comparative evaluation, these benchmarks do not cap-

ture the reliability requirements of safety-critical deployment. A model that achieves high GQA accuracy may still produce catastrophically wrong outputs in emergency or disaster scenarios.

The Top-N Robot Selection task, used as the primary evaluation in this thesis, is a multi-label morphology classification problem: given a scene image and mission description, predict which robot types (Drone, Underwater Robot, Humanoid, Wheeled, Legged) should be deployed. This task is mission-critical because wrong predictions directly lead to incorrect asset allocation in emergency response scenarios.

VERI-Emergency [26], introduced at WACV 2026, is a visual emergency recognition benchmark covering fire, flooding, chemical hazards, and related emergency scenarios. It introduces an asymmetric evaluation metric structure distinguishing OverRx (false positive rate: wrongly flagging non-emergencies) from UnderRx (false negative rate: missing genuine emergency signals). For safety-critical deployment, UnderRx is the more consequential metric: a missed emergency is more dangerous than a false alarm.

GEO-Bench-VLM [27], introduced at ICCV 2025, is a geospatial disaster understanding benchmark covering satellite and aerial imagery of flood zones, wildfire damage, earthquake devastation, and infrastructure collapse. It evaluates models on fine-grained scene classification tasks requiring both visual recognition and geographical contextual reasoning.

Together, these three benchmarks provide a more realistic and demanding evaluation surface for edge-deployable VLMs than standard generative benchmarks. EmberVLM achieves state-of-the-art performance across all three among sub-billion-parameter models, while simultaneously meeting

strict edge-deployment latency and memory constraints.

2.6 Low-Carbon and Sustainable AI

The environmental cost of training large AI models has attracted increasing attention. Strubell et al. [28] estimated that training a large NLP model from scratch produces carbon emissions comparable to five automobile lifetimes. More recent estimates place the training carbon footprint of GPT-3 at approximately 552 tonnes CO₂eq [29]. As VLMs grow to hundreds of billions of parameters, their environmental impact grows correspondingly.

Compact, parameter-efficient model design is therefore not only a systems engineering concern but an environmental imperative. By constraining total trainable parameters to under 40M and completing all training stages on a dual-A100 setup in approximately 90 hours, EmberVLM achieves a total measured training footprint of 25.3 kg CO₂eq, verified using the CodeCarbon [30] framework. This is more than 21,000 times smaller than GPT-3’s estimated footprint, demonstrating that mission-critical capable multimodal AI can be trained sustainably. This thesis treats low-carbon training not as a secondary benefit but as a primary design constraint.

Chapter 3 BitMar: Low-Bit

Multimodal Foundation for Edge

Deployment

This chapter presents BitMar [6], the first published contribution of this thesis research. BitMar is framed as a proof-of-concept and foundational work that established key design principles, notably the compatibility of extreme quantization with cross-modal fusion and episodic memory conditioning, which were later refined and scaled in the EmberVLM system (Chapter 4).

3.1 Motivation and Research Goals

The design of BitMar is motivated by a specific question: *Can a model with only 14 million parameters, operating under 1.58-bit ternary weight quantization, achieve meaningful multimodal understanding and generation while incorporating an external episodic memory?*

This question is non-trivial because each component introduces constraints on the others. Quantization reduces the information capacity per parameter, potentially limiting the model’s ability to learn fine-grained cross-modal correspondences. Cross-modal fusion requires the text and vision representations to be sufficiently expressive to support accurate attention-based alignment despite the quantization noise. Episodic memory adds a fixed-overhead read/write mechanism that must integrate cleanly with the quantized decoder without introducing instability or gradient conflicts.

Why 1.58-bit? Ternary weight quantization at 1.58 bits/parameter enables replacing floating-point multiply-accumulate operations with additions and subtractions. A ternary weight $w \in \{-1, 0, +1\}$ applied to activation a gives: $w \cdot a \in \{-a, 0, +a\}$. This means every MHSA and FFN projection can be computed without a single floating-point multiplication, with only conditional negation or zeroing. On hardware without dedicated floating-point units, this is a substantial speedup. Even on standard CPUs, integer arithmetic is significantly faster than floating-point, making ternary quantization the preferred precision for the target edge hardware.

The three design goals for BitMar are:

1. **Unified low-bit multimodal encoding:** both text and vision representation pathways operate under the same 1.58-bit quantization regime, ensuring consistent hardware efficiency across the entire model.
2. **Cross-modal fusion under quantization constraints:** the fusion mechanism must preserve semantic alignment between modalities despite ternary weight projections, validated by a contrastive cross-modal alignment objective.
3. **External episodic memory without parametric overhead:** the memory system must improve contextual coherence and task accuracy without significantly increasing the trainable parameter count, and must not require any additional inference hardware.

3.2 BitMar Architecture Overview

BitMar follows a four-stage pipeline, illustrated in Figure 3.1:

1. **Parallel low-bit encoders:** A 1.58-bit BitNet-style text encoder and a DiNOv2-based vision encoder (with MLP compression) produce 128-dimensional representations of the input text and image respectively.
2. **Cross-modal fusion:** A lightweight cross-attention module (text queries, vision keys/values) fuses the two modalities into a shared 128-dimensional representation.
3. **Episodic memory conditioning:** The fused multimodal embedding queries a fixed-size episodic memory matrix ($K=512$, $C=128$), retrieving a context vector that conditions the decoder.
4. **Memory-conditioned autoregressive decoding:** A 1.58-bit BitNet decoder generates text output, with per-layer conditioning from the retrieved memory vector.

All trainable weight matrices in the text encoder, fusion module, and decoder use ternary weights $\{-1, 0, +1\}$ with learned per-layer scales. Activations use 8-bit quantization with per-token max-abs scaling. Softmax, residual connections, and layer normalization operate in FP32 to preserve numerical stability.

3.3 Text Encoder

3.3.1 Architecture

The BitMar text encoder is a lightweight quantized Transformer with 4 layers, hidden dimension $d=128$, and $h=4$ attention heads, supporting

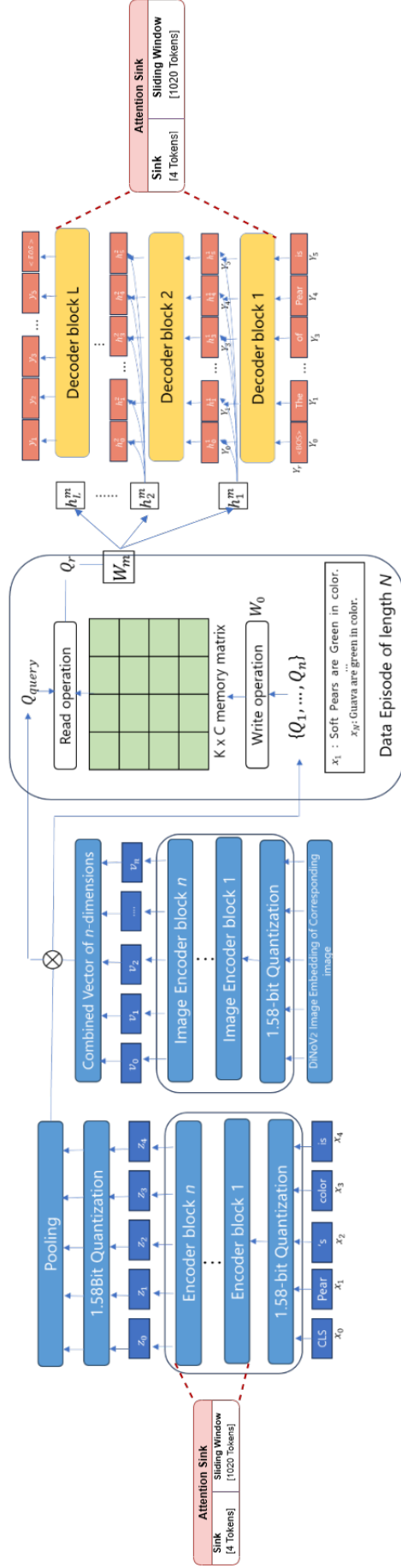


Figure 3.1: BitMar Architecture. The model processes multimodal inputs: text tokens and DiNov2-compressed image features. Quantized encoders (1.58-bit) generate compact text and vision embeddings (z, v), which are fused via cross-modal attention into shared query representations. A sliding-window attention mechanism enables long-context processing. A fixed episodic memory matrix ($K \times C$) stores and retrieves multimodal context vectors through quantized read/write weights, supporting optional SD-card offloading for edge deployment.

sequences of up to 256 tokens.

3.3.2 Quantization

Two levels of quantization are applied:

Weight Quantization. All linear projection weights in MHSA and FFN sublayers are quantized to ternary values $\{-1, 0, +1\}$ using learned per-layer scaling factors γ :

$$\hat{W} = \text{RoundClip}\left(\frac{W}{\gamma + \epsilon}, -1, 1\right), \quad \gamma = \frac{1}{nm} \sum_{i,j} |W_{ij}| \quad (3.1)$$

where $\text{RoundClip}(\cdot, a, b) = \max(a, \min(b, \text{round}(\cdot)))$ maps each weight to the nearest ternary value. The scale γ is the mean absolute value of the unquantized weight matrix, trained jointly with the network parameters.

Activation Quantization. Token-wise activations are quantized to 8 bits using per-token max-abs scaling:

$$\hat{\mathbf{x}}_t = \text{round}\left(\frac{\mathbf{x}_t}{\max_j |x_{t,j}|} \times 127\right) \quad (3.2)$$

This localized scaling preserves fine-grained detail for each token by adapting to that token’s specific activation range.

3.3.3 Attention Sinks for Long-Context Processing

Attention sinks are injected into every self-attention layer to enable efficient long-sequence processing. With $S=4$ persistent sink tokens and a sliding window of $W=1,020$, the KV cache maintains global context anchors

alongside recent tokens. When a new token arrives, the oldest window token is evicted, sink and window sets are merged, and positional indices are clamped to $[0, S + W - 1]$, yielding $O(S + W)$ memory regardless of sequence length.

3.4 Vision Encoder

The vision branch leverages pretrained DiNOv2 [31] to extract 768-dimensional patch features *offline*, avoiding heavy backbone inference at deployment time. 2×2 average pooling reduces the patch count by $4 \times$. A learned two-layer MLP bottleneck then compresses $768 \rightarrow 128$ dimensions:

$$\mathbf{z}_{\text{vis}}^{(1)} = \text{ReLU}(\mathbf{W}_1 \mathbf{x}_{\text{vis}} + \mathbf{b}_1), \quad (3.3)$$

$$\mathbf{z}_{\text{vis}}^{(2)} = \mathbf{W}_2 \mathbf{z}_{\text{vis}}^{(1)} + \mathbf{b}_2 \quad (3.4)$$

where $\mathbf{W}_1 \in \mathbb{R}^{384 \times 768}$, $\mathbf{W}_2 \in \mathbb{R}^{128 \times 384}$. All subsequent fusion, memory, and decoder pathways operate in the 128-dimensional space.

3.5 Cross-Modal Fusion

Given pooled text tokens $\mathbf{Z} \in \mathbb{R}^{n_t \times 128}$ and compressed vision tokens $\mathbf{V}_{\text{img}} \in \mathbb{R}^{n_v \times 128}$, the fusion module applies standard cross-attention [32] with text queries and vision keys/values:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V} \quad (3.5)$$

All projection matrices use 1.58-bit ternary weights; softmax and residual/LN run in FP32. The fused output $\mathbf{F} \in \mathbb{R}^{n_t \times 128}$ is mean-pooled to a single

query vector $\mathbf{q}_{\text{mem}} \in \mathbb{R}^{128}$ for memory retrieval.

An InfoNCE contrastive loss [33] enforces cross-modal alignment:

$$\mathcal{L}_{\text{cm}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(\mathbf{z}_t^i, \mathbf{z}_v^i)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(\mathbf{z}_t^i, \mathbf{z}_v^j)/\tau)} \quad (3.6)$$

3.6 Episodic Memory Module

3.6.1 Memory Matrix Specification

The memory is a learnable matrix $\mathbf{M} \in \mathbb{R}^{K \times C}$ with $K=512$ slots and $C=128$ dimensions. Each slot stores a 128-dimensional latent vector summarizing multimodal context from past inputs.

3.6.2 Writing Mechanism

At step t , query $\mathbf{q}_t \in \mathbb{R}^C$ is written to the memory via learned write weights $\mathbf{W}_w \in \mathbb{R}^K$ using a soft outer-product update:

$$\mathbf{M} \leftarrow \mathbf{M} + \alpha \cdot \mathbf{W}_w \mathbf{q}_t^\top \quad (3.7)$$

where $\alpha=0.2$. This distributes the write proportionally across all slots, ensuring stability.

3.6.3 Reading Mechanism

Content-based addressing [17] produces the retrieved vector:

$$\mathbf{W}_r = \text{softmax}(\mathbf{M} \mathbf{q}_t) \in \mathbb{R}^K, \quad (3.8)$$

$$\mathbf{M}_r = \mathbf{W}_r^\top \mathbf{M} \in \mathbb{R}^{1 \times C} \quad (3.9)$$

3.6.4 Regularization and Forgetting

A Frobenius-norm penalty on successive updates prevents thrashing:

$$\mathcal{L}_{\text{mem}} = \lambda \|\Delta \mathbf{M}\|_F^2, \quad \Delta \mathbf{M} = \mathbf{M}^{(t)} - \mathbf{M}^{(t-1)} \quad (3.10)$$

Usage-based forgetting additionally attenuates stale slots.

Algorithm 1 BitMar episodic memory read/write procedure.

Require: Query $\mathbf{q}_t \in \mathbb{R}^C$, Memory $\mathbf{M} \in \mathbb{R}^{K \times C}$,

write weights $\mathbf{W}_w \in \mathbb{R}^K$, write rate α , regularization weight λ

Ensure: Updated memory $\mathbf{M}$, retrieved context vector $\mathbf{M}_r$

- 1: **Write:** $\mathbf{M} \leftarrow \mathbf{M} + \alpha \cdot \mathbf{W}_w \mathbf{q}_t^\top$
 - 2: **Read weights:** $\mathbf{W}_r \leftarrow \text{softmax}(\mathbf{M} \mathbf{q}_t)$
 - 3: **Retrieve:** $\mathbf{M}_r \leftarrow \mathbf{W}_r^\top \mathbf{M}$
 - 4: **Regularize:** $\mathcal{L}_{\text{mem}} += \lambda \|\mathbf{M}^{(t)} - \mathbf{M}^{(t-1)}\|_F^2$
 - 5: **return** $\mathbf{M}, \mathbf{M}_r$
-

3.7 Decoder with Attention Sinks

The decoder is a 4-layer causal Transformer ($d=128$, $h=4$, max 256 tokens) with the same attention sink mechanism as the encoder. At each step, the retrieved memory vector $\mathbf{M}_r$ is projected and injected as per-layer conditioning. The output projection ($128 \rightarrow 50,257$ GPT-2 vocabulary) uses ternary weights; logit computation runs in FP32.

3.8 Training Objectives and Adaptive Curriculum

3.8.1 Multi-Component Objective

The total training objective combines three terms:

$$\mathcal{L} = \mathcal{L}_{\text{lm}} + 1.5 \cdot \mathcal{L}_{\text{cm}} + 0.1 \cdot \mathcal{L}_{\text{mem}} \quad (3.11)$$

where $\mathcal{L}_{\text{lm}}$ is the standard language modeling cross-entropy, $\mathcal{L}_{\text{cm}}$ is the InfoNCE cross-modal alignment loss (weighted 1.5 to prioritize alignment), and $\mathcal{L}_{\text{mem}}$ is the memory consistency regularizer (weighted 0.1 as a light stabilizer).

3.8.2 Adaptive Training Controller

The Adaptive Training Controller (ATC) monitors a 200-step EMA of cross-modal cosine similarity. When the EMA drops > 0.12 from its recent maximum (with ≥ 800 -step cooldown), it randomly freezes one encoder or upweights $\mathcal{L}_{\text{cm}}$ for 1,500 steps, preventing modality collapse. Algorithm 2 formalizes this procedure.

3.9 Dataset and Training Configuration

Training uses a 100M-token corpus: 50M multimodal tokens from CC3M [34] and Localized Narratives [35] (with precomputed DiNOv2 features), and 50M text-only tokens from the BabyLM 2025 corpus [36] spanning BNC, CHILDES, Gutenberg, OpenSubtitles, Simple English Wikipedia,

Algorithm 2 Adaptive Training Controller (ATC) for modality collapse prevention.

Require: EMA window $W_{\text{ema}}=200$, drop threshold $\delta=0.12$,

freeze duration $T_f=1,500$, cooldown $T_c=800$

Require: Cross-modal cosine similarity at step t :

$$s_t = \cos(\mathbf{z}_t^{\text{text}}, \mathbf{z}_t^{\text{vis}})$$

```

1:  $\bar{s}_0 \leftarrow s_0$ ;  $s_0^* \leftarrow s_0$ ; last_intervention  $\leftarrow -T_c$ 
2: for each training step  $t$  do
3:    $\bar{s}_t \leftarrow (1 - 1/W_{\text{ema}}) \bar{s}_{t-1} + (1/W_{\text{ema}}) s_t$  ▷ update EMA
4:    $s_t^* \leftarrow \max(s_{t-1}^*, \bar{s}_t)$  ▷ track peak EMA
5:   if  $\bar{s}_t < s_t^* - \delta$  and  $t - \text{last\_intervention} \geq T_c$  then
6:     Sample  $a \sim \text{Uniform}\{\text{freeze\_text}, \text{freeze\_vision}, \text{boost\_cm}\}$ 
7:     if  $a = \text{freeze\_text}$  then
8:       Freeze text encoder parameters for  $T_f$  steps
9:     else if  $a = \text{freeze\_vision}$  then
10:      Freeze vision MLP parameters for  $T_f$  steps
11:    else
12:      Set  $w_{\text{cm}} \leftarrow 3.0$  for  $T_f$  steps, then restore to 1.5
13:    end if
14:    last_intervention  $\leftarrow t$ 
15:  end if
16:  Update via AdamW:  $\mathcal{L} = \mathcal{L}_{\text{lm}} + w_{\text{cm}} \mathcal{L}_{\text{cm}} + 0.1 \mathcal{L}_{\text{mem}}$ 
17: end for

```

and Switchboard. A 1M-token hold-out tracks alignment and perplexity throughout training. Training configuration is summarized in Table 3.1.

3.10 Quantization Effectiveness Analysis

The quantization effectiveness metric E_q measures the fraction of ternary weights set to zero across all quantized layers. A higher E_q indicates greater

Table 3.1: BitMar training configuration. Key hyperparameters used during the 10-epoch training run on the 100M-token corpus.

Hyperparameter	Value
Text encoder layers	4
Hidden dimension d	128
Memory slots K	512
Memory dimension C	128
Sink tokens S	4
Sliding window W	1,020
Cross-modal loss weight	1.5
Memory consistency weight	0.1
ATC drop threshold	0.12
ATC freeze duration (steps)	1,500
ATC minimum interval (steps)	800
Optimizer	AdamW8bit
Learning rate	2×10^{-4}
Effective batch size	128
Precision	FP16
Training hardware	NVIDIA A6000

effective compression. As shown in Figure 3.2, E_q stabilizes at 42.8% by the end of training, demonstrating that BitMar learns dense representations within its compressed ternary space. Early growth (0–200k steps) is step-like as encoders adapt; after 400k steps, growth smooths toward convergence.

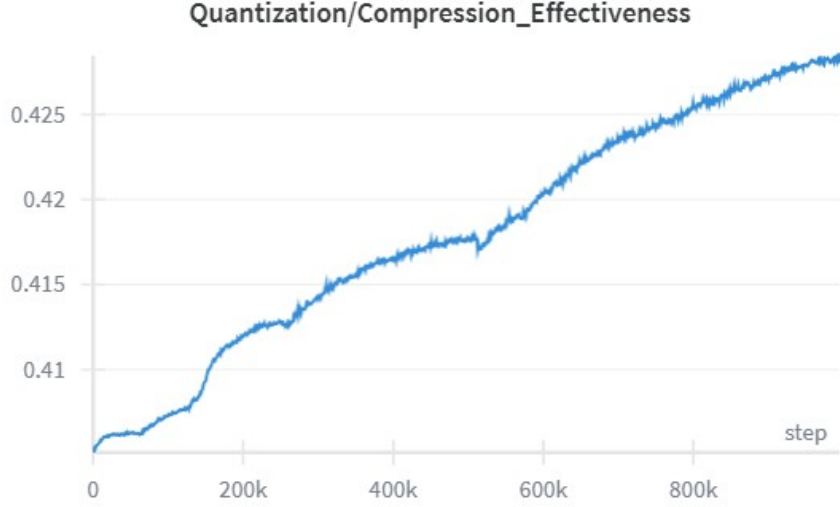


Figure 3.2: Quantization effectiveness over training. E_q (fraction of ternary weights set to zero) stabilizes at 42.8%, demonstrating efficient compression without downstream performance degradation.

3.11 Experimental Results

3.11.1 Language Understanding Benchmarks

Table 3.2: Language understanding benchmark performance. ✓ = natively 1-bit trained. Scores are accuracy (%). WG = WinoGrande, CQA = CommonsenseQA, ARC-E = ARC-Easy.

Model	Params	1-bit	ARC-E	BoolQ	HSwag	WG	CQA	MMLU
Bonsai 0.5B	500M	✓	58.25	58.44	48.01	54.46	18.43	25.74
OLMo-BitNet 1B	1B	✓	25.38	52.48	25.88	51.54	19.49	25.47
Falcon3-1.58bit 7B	7B	×	65.03	72.14	59.46	60.14	67.08	42.79
LLaMA3-8B-1.58	8B	×	70.71	68.38	68.56	60.93	28.50	35.04
BitNet b1.58 2B	2B	✓	74.79	80.18	68.44	71.90	71.58	53.17
BitMar-14M (Ours)	14M	✓	28.32	42.83	30.04	54.57	24.57	27.90

Table 3.2 compares BitMar-14M against low-bit quantized baselines. Despite being $35\times$ smaller than the next-smallest baseline (Bonsai 0.5B),

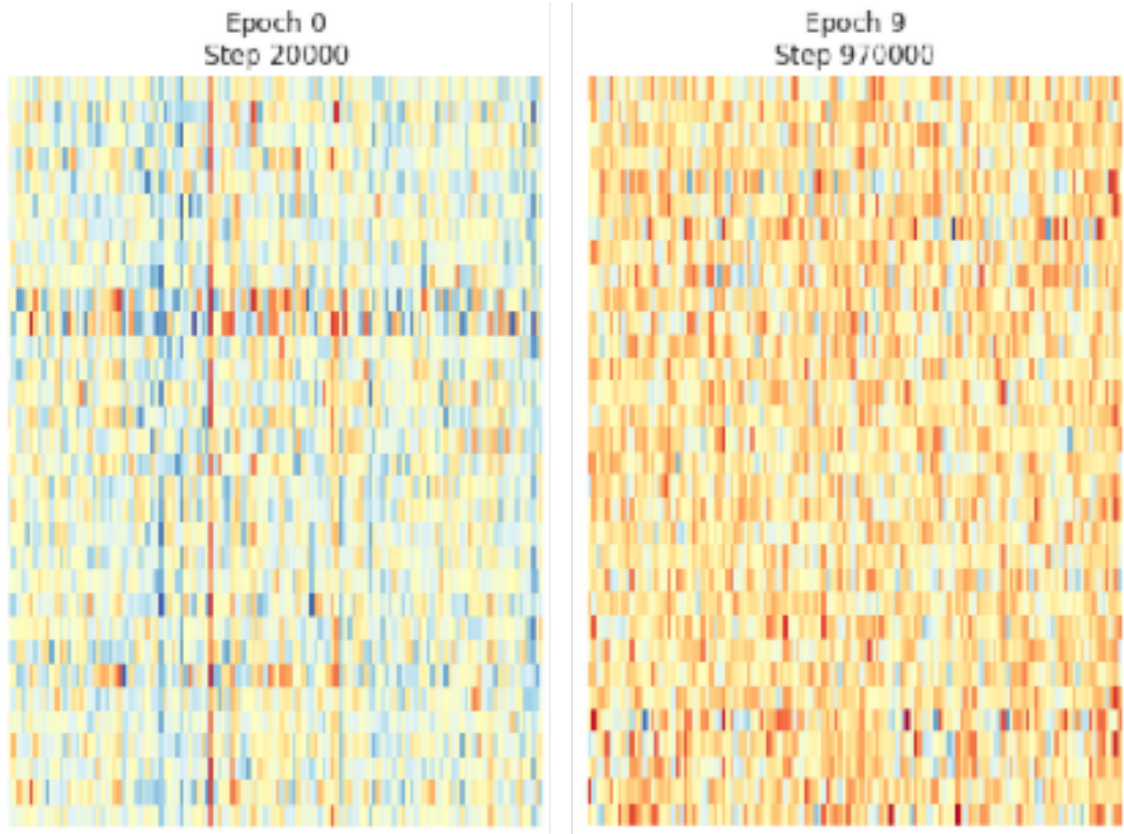


Figure 3.3: Episodic memory slot activation patterns across training checkpoints.

Each row corresponds to a different checkpoint, ranging from early training (top, Epoch 0, step 20k) to late training (bottom, Epoch 9, step 97k). Early checkpoints show scattered, weak activations with minimal slot specialization. By the final checkpoint, memory slots develop strongly differentiated activation profiles, with specific slots specializing for distinct semantic categories. This convergence confirms that the episodic memory organizes stored representations by semantic content rather than recency.

BitMar achieves competitive BoolQ (42.83%) and WinoGrande (54.57%), demonstrating strength in binary reasoning and coreference. ARC-Easy (28.32%) and HellaSwag (30.04%) trail larger models, reflecting capacity limits on multi-step commonsense reasoning.

Table 3.3: BitMar BabyLM evaluation. Average finetuned NLP accuracy: 60.5%.

Category	Task	Metric	Score
Finetune NLP	BoolQ	Accuracy	66.5%
	MNLI	Accuracy	42.3%
	MRPC	Accuracy	69.1%
	MultiRC	Accuracy	57.6%
	QQP	Accuracy	70.2%
	RTE	Accuracy	54.0%
	WSC	Accuracy	63.5%
Multimodal	DevBench	Visual Vocab Acc.	21.2%
	VQA	Accuracy	21.4%
	Winoground	Accuracy	23.8%
World Knowledge	EWoK	Accuracy	24.9%
Linguistic	BLiMP	Accuracy	48.7%
	Compositional	Accuracy	51.5%
	Entity Tracking	Accuracy	31.2%
Psycholinguistic	Reading Comp.	Score	0.44
Morphology	Wug Adj.	Correlation	−0.16
	Wug Past	Correlation	−0.22

3.11.2 BabyLM Evaluation Suite

BitMar achieves 60.5% average across finetuned NLP benchmarks, with strengths in paraphrase detection (QQP: 70.2%, MRPC: 69.1%) and reading comprehension (BoolQ: 66.5%). Multimodal tasks score 21–25%,

with the highest result on EWoK (24.9%), plausibly due to episodic memory providing relevant context. Negative Wug morphology correlations (-0.16 , -0.22) indicate systematic limits in morphological productivity under extreme compression.

3.11.3 Episodic Memory Ablation

Table 3.4: Inference efficiency with and without episodic memory.

Metric	Memory On	Memory Off
Throughput (tok/s)	57.3	7.7
Latency/token (ms)	17.3	129.8
Energy (J)	1.90	9.17
RAM (MB)	956	1,076

Table 3.5: Zero-shot accuracy deltas (Δ pp) with episodic memory enabled.

Task	Δ (pp)
Entity Tracking (Split 1)	+2.9
Entity Tracking (Split 2)	+4.1
COMPS	+3.4
BLiMP	+0.6
VQA	+3.4
EWoK (Split 1)	-1.6
EWoK (Split 2)	+1.0
Winoground	-1.6
DevBench	No effect

Episodic memory provides $7.5\times$ throughput improvement, 79% energy reduction, and 11% lower RAM usage. Zero-shot accuracy gains of 3–4 pp are consistent across entity tracking and visual question answering. Two regressions (Winoground -1.6pp , EWoK Split 1 -1.6pp) are attributed to the fixed-capacity memory occasionally introducing irrelevant context, a tradeoff inherent in any fixed-size retrieval system.

3.12 Implementation Details

3.12.1 Hyperparameters

Table 3.6 lists the architectural hyperparameters and Table 3.7 lists the training configuration.

Table 3.6: BitMar architectural hyperparameters. Full specification of the 14M-parameter ternary-quantized architecture.

Hyperparameter	Value
<i>Text Encoder</i>	
Transformer layers	4
Hidden dimension d	128
Attention heads h	4
Max sequence length	256 tokens
Weight quantization	Ternary $\{-1, 0, +1\}$, learned per-layer scale
Activation quantization	Per-token 8-bit max-abs, range $[-127, 127]$

Continued on next page

Table 3.6 (continued)

Hyperparameter	Value
Attention sinks S	4
Sliding window W	1,020
<i>Vision Encoder</i>	
Vision backbone	DiNOv2 (frozen, offline feature extraction)
DiNOv2 feature dimension	768
Spatial pooling	2×2 average pooling
MLP compression $\mathbf{W}_1$	$\mathbb{R}^{384 \times 768}$
MLP compression $\mathbf{W}_2$	$\mathbb{R}^{128 \times 384}$
Output dimension	128
<i>Cross-Modal Fusion</i>	
Fusion type	Cross-attention (text queries, vision K/V)
Projection quantization	1.58-bit ternary
Fusion output	$\mathbf{F} \in \mathbb{R}^{n_t \times 128}$
Memory query pooling	Mean pooling $\rightarrow \mathbf{q}_{\text{mem}} \in \mathbb{R}^{128}$
<i>Episodic Memory</i>	
Memory slots K	512
Memory dimension C	128
Write rate α	0.2
Memory regularization λ	0.1

Continued on next page

Table 3.6 (continued)

Hyperparameter	Value
Addressing mechanism	Content-based (softmax inner product)
<i>Decoder</i>	
Transformer layers	4
Hidden dimension	128
Attention heads	4
Max generation length	256 tokens
Attention sinks	Same as encoder
Output vocabulary	GPT-2 BPE (50,257 tokens)
Output projection	Ternary weights, FP32 logits

3.12.2 Training Corpus

Table 3.8 details the composition of BitMar’s 101M-token training corpus. Multimodal pairs (50M tokens) from CC3M and Localized Narratives provide visual-language grounding; text-only data (50M tokens) from six sources provides linguistic coverage. A 1M-token stratified hold-out set is reserved for evaluation.

3.12.3 Tokenizer and Preprocessing

BitMar uses the GPT-2 Byte Pair Encoding (BPE) tokenizer with a vocabulary size of 50,257 tokens. Text sequences exceeding 256 tokens

Table 3.7: BitMar training hyperparameters. Optimizer, schedule, memory, and hardware settings used across the full 10-epoch run.

Hyperparameter	Value
<i>Training</i>	
Optimizer	AdamW8bit
Learning rate	2×10^{-4}
Cosine restart T_0	1,000 steps
Cosine restart T_{mult}	2
Min learning rate η_{min}	$0.1 \times \text{lr}$
Effective batch size	128
Training epochs	10
Precision	FP16
Gradient checkpointing	Yes
Training hardware	NVIDIA A6000
<i>Loss Weights</i>	
Language modeling $\mathcal{L}_{\text{lm}}$	1.0
Cross-modal contrastive $\mathcal{L}_{\text{cm}}$	1.5
Memory consistency $\mathcal{L}_{\text{mem}}$	0.1
<i>Adaptive Training Controller</i>	
EMA window	200 steps
Drop threshold	0.12
Freeze/boost duration	1,500 steps
Min intervention interval	800 steps

Table 3.8: BitMar training corpus composition. The 101M-token corpus combines 50M multimodal tokens and 50M text-only tokens, with a 1M-token stratified hold-out set.

Type	Source	Tokens
Multimodal (50M)	CC3M (Conceptual Captions)	~28M
	Localized Narratives	~22M
Text-only (50M)	BNC Spoken	~10M
	CHILDES	~5M
	Gutenberg	~12M
	OpenSubtitles	~10M
	Simple English Wikipedia	~5M
	Switchboard	~8M
Hold-out	Mixed (stratified)	1M
Total		101M

are truncated; shorter sequences are padded. DiNOv2 visual features are extracted offline using a frozen DiNOv2 (large) backbone, storing 2×2 -pooled 768-dimensional patch features as memory-mapped .npy files. During training, features are loaded with on-the-fly compression to reduce I/O overhead. Each multimodal sample is an (image, caption) pair, with the caption tokenized using the GPT-2 BPE tokenizer.

3.12.4 External Memory Offloading

For memory-constrained devices, the memory matrix $\mathbf{M}$ can be offloaded to external storage via a utility that saves and loads memory tensors

without gradient tracking. This preserves the persistent memory state between sessions without increasing runtime RAM usage. The offload is triggered automatically when device RAM drops below a configurable threshold, making BitMar deployable on hardware with as little as 512 MB RAM.

3.13 Lessons Learned and Research Directions

BitMar validates three design hypotheses that directly motivated EmberVLM:

Validated: episodic memory is compatible with and beneficial for quantized multimodal models. The efficiency gains and accuracy improvements confirm that episodic memory can replace heavy long-range attention even under aggressive 1.58-bit quantization.

Validated: 1.58-bit quantization is compatible with cross-modal fusion. BabyLM multimodal task performance demonstrates that ternary weights support meaningful cross-modal alignment at 14M parameters.

Open: Can episodic memory address hallucination? BitMar’s memory improves coherence but has no mechanism for detecting visually ungrounded tokens. This gap directly motivated the Visual Absence Refiner in EmberVLM (Section 4.6).

Open: Can the architecture scale to mission-critical classification? BitMar’s evaluation is primarily generative. Whether memory-augmented tiny

VLMs can achieve reliable multi-label classification in safety-critical settings required a dedicated training pipeline and evaluation framework: the EmberVLM design.

Open: Can memory be truly adaptive at deployment? BitMar’s memory is essentially static at deployment: slots do not update after training. EmberVLM’s novelty-triggered online writes (Section 4.5.5) directly address this gap.

Chapter 4 **EmberVLM: Efficient**

Edge Vision-Language Model

This chapter presents EmberVLM, the primary architectural contribution of this thesis. EmberVLM is a 164M total parameter (40M trainable) tiny VLM purpose-built for reliable mission-critical inference under strict edge-deployment constraints. It builds directly on the design principles established in BitMar (Chapter 3), scaling to a more capable backbone while introducing two novel modules: the Episodic Memory Module and the Visual Absence (VA) Refiner.

4.1 Overview and Design Philosophy

Figure 4.1 illustrates the complete EmberVLM architecture. EmberVLM is guided by three design principles:

1. **Compositional efficiency:** separate frozen pretrained representations (DINOv2-Small for visual perception, SmolLM-135M for linguistic priors) from a compact trainable core (40M parameters). This preserves the general capabilities of large pretraining while keeping training cost and carbon emissions minimal.
2. **Memory-augmented reliability:** treat episodic memory not as a static knowledge base but as a dynamically loadable, deployment-time cache that provides contextual biases drawn from prior field experience. Memory breaks the traditional latency–accuracy trade-off by providing near-zero-cost retrieval from a compact on-device

matrix.

3. **Mechanistic hallucination control:** instead of suppressing hallucinations through additional supervision or retraining, identify and penalize visually ungrounded tokens at inference time by monitoring the model’s own internal neuron-level activation patterns.

4.2 Base Multimodal Pipeline

4.2.1 Vision Encoder

EmberVLM employs DINOv2-Small [31] as the frozen visual perception module (21.1M parameters). The encoder aggressively compresses an input image X_v into a minimal sequence of $N_v=8$ visual tokens with feature dimension $d_{\text{vis}}=384$. DINOv2-Small’s self-supervised training provides robust visual features without task-specific supervision, and freezing it throughout training preserves general visual priors while contributing zero parameters to the trainable budget.

4.2.2 Language Backbone

SmolLM-135M [37] serves as both the text encoder and the language modeling backbone (135M parameters). SmolLM-135M is frozen in Stages 1, 3, and 4, and fully unfrozen end-to-end in Stage 2 (Multimodal Instruction Tuning). This staged approach allows the model to first learn cross-modal alignment while preserving linguistic priors, then adapt the full backbone to complex multimodal instruction following.

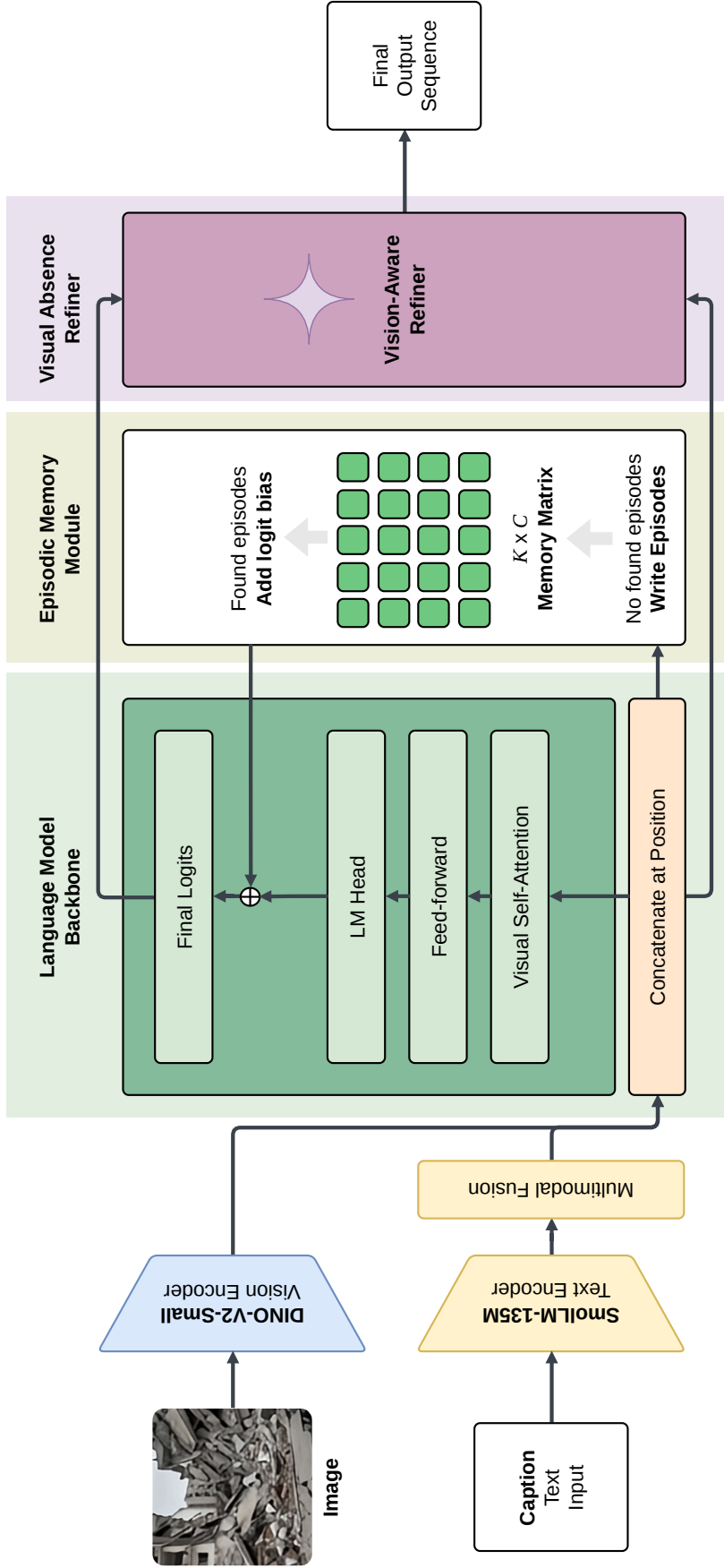


Figure 4.1: EmberVLM architecture overview. The system integrates a frozen DINOv2-Small vision encoder, a SmolLM-135M language backbone, a Stable Gated Fusion module, an Episodic Memory Module, and the Visual Absence (VA) Refiner. Solid arrows denote forward-pass data flow; the memory matrix is outside the gradient graph and is updated algebraically at deployment time.

4.2.3 Parameter Breakdown

Figure 4.2 illustrates the distribution of parameters across components. Of the 164M total parameters, only 40M (24.4%) are updated during training; the remaining 75.6% are frozen pretrained representations that contribute visual and linguistic priors at zero training cost.

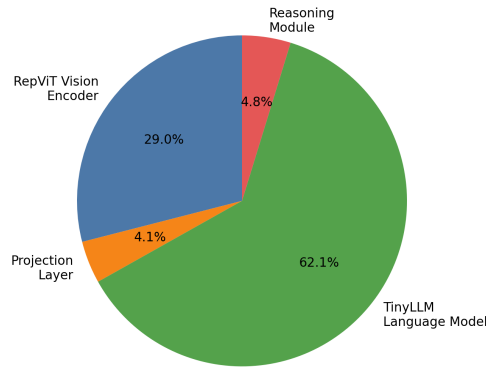


Figure 4.2: EmberVLM parameter distribution. DINOv2-Small (21.1M) and the frozen layers of SmolLM-135M account for the majority of total parameters and are never updated. The 40M trainable parameters are concentrated in the Fusion Module, bottleneck adapter, and Reasoning Head.

Table 4.1: EmberVLM parameter breakdown. Total: 164M parameters, 40M trainable.

Component	Params (M)	Trainable	Frozen Stages
DINOv2-Small (vision encoder)	21.1	—	All stages
SmolLM-135M (text enc. + backbone)	135.0	Partial	1, 3, 4
Fusion Module (cross-attn + gate)	7.27	✓	None
Bottleneck adapter	0.06	✓	None
Reasoning & Action Head	0.57	✓	1, 2
Total	164.0	40M	—

Table 4.2: EmberVLM architectural specification. Component-level summary of the 164M-parameter system, including encoder, backbone, memory, and VA Refiner configuration.

Component	Specification
Vision encoder	DINOv2-Small (21.1M, frozen)
Visual tokens N_v	8
Visual feature dim d_{vis}	384
Language backbone	SmolLM-135M (135M)
Language feature dim d_{lang}	576
Bottleneck adapter dim	48
Vision gate initialization α	2.5
Episodic memory slots K	512
Episodic memory dim C	576
Memory size (fp32)	≈ 1.2 MB
VA Refiner hook layers	10–14 (SmolLM-135M FFN)
VA Refiner neuron count	128 (top- k per layer)
VA Refiner λ	0.5
Visual threshold p_{VA}	0.6
Non-visual threshold p_{VA}	0.8
Latent CoT temperature τ	0.07

4.3 Stable Gated Fusion

Raw visual tokens from DINOv2-Small (dimension 384) are projected into the language backbone’s token embedding space ($d_{\text{lang}}=384$) via a parameter-efficient bottleneck adapter ($d_{\text{bottleneck}}=48$). A learnable vision gate scalar α (initialized to 2.5) dynamically scales the adapted visual in-

fluence:

$$V_{\text{fused}} = \sigma(\alpha) \cdot \text{LayerNorm}(V_{\text{adapt}}) \quad (4.1)$$

where σ denotes the sigmoid activation. At initialization, $\sigma(2.5) \approx 0.92$, so visual influence starts near-full and becomes learnable rather than fixed. This gate allows the model to calibrate how much visual influence to apply during alignment learning, preventing early-stage visual features from overwhelming linguistic priors.

The gated visual tokens are interleaved with textual embeddings to form the joint context sequence processed by the language backbone.

4.4 Task-Specific Adaptation: Top- K Robot Selection Head

While EmberVLM supports standard autoregressive generation, autoregressive decoding is inefficient and error-prone for mission-critical tactical deployment. For the primary robot selection task, the standard language modeling head is bypassed. Instead, final hidden states from the language backbone are mean-pooled to produce a dense incident representation $\mathbf{h}_{\text{pool}}$. A *Reasoning and Action Head* MLP maps $\mathbf{h}_{\text{pool}}$ to a probability distribution over the robot fleet (Drone, Underwater Robot, Humanoid, Wheeled, Legged):

$$\mathbf{p}_{\text{robot}} = \text{softmax}(\text{MLP}(\mathbf{h}_{\text{pool}})) \quad (4.2)$$

This framing decouples general scene comprehension from tactical execution. Avoiding autoregressive decoding eliminates the token-by-token latency overhead, keeping total trainable parameters strictly under 40M.

4.5 Episodic Memory Module

4.5.1 Overview and Distinction from BitMar

EmberVLM’s episodic memory differs fundamentally from BitMar’s gradient-trained memory. BitMar’s memory matrix is updated via back-propagation and is static at deployment. EmberVLM’s memory has three key properties:

- **Outside the gradient graph:** the memory matrix is not differentiated through, as confirmed in the supplementary material.
- **Pre-populated offline:** populated from the training split during a READ_ONLY pre-population phase before evaluation.
- **Updatable online at deployment:** in ONLINE_UPDATE mode, new embeddings are written to memory during inference without weight updates, enabling adaptation to novel scenarios.

4.5.2 Memory Matrix Specification

The memory matrix $\mathbf{M} \in \mathbb{R}^{K \times C}$ has $K=512$ slots and $C=576$ feature dimensions, alongside an associated covariance matrix. Physical size: $512 \times 576 \times 4 \text{ bytes} \approx 1.2 \text{ MB (fp32)}$. The dimension $C=576$ matches the SmolLM-135M token embedding dimension, allowing direct residual injection of retrieved memory into the generation process without additional projection overhead.

4.5.3 Scope Detector

Before engaging memory retrieval, the incoming multimodal embedding $\mathbf{z}$ is evaluated by a lightweight Scope Detector: a 2-layer MLP binary classifier outputting $p(\text{in-scope})$. If $p < 0.5$, the query is deemed out-of-scope and the model defaults to parametric generation without memory. If $p \geq 0.5$, the fast memory retrieval path activates.

The scope detector serves as a gating mechanism that prevents irrelevant historical episodes from contaminating queries about novel or out-of-distribution scenes. Without scope detection, the memory module would apply retrieved context to every query, potentially degrading performance on genuinely novel scenarios.

4.5.4 Memory Addressing and Retrieval

When in-scope, normalized addressing weights w_k over all K memory slots are computed via Gaussian distance-based addressing:

$$w_k = \frac{\exp\left(-\frac{\|\mathbf{z} - \mathbf{M}_k\|^2}{2\alpha\nu\tau}\right)}{\sum_{j=1}^K \exp\left(-\frac{\|\mathbf{z} - \mathbf{M}_j\|^2}{2\alpha\nu\tau}\right)} \quad (4.3)$$

where α, ν, τ represent the update strength, variance, and temperature scaling factors, respectively. The retrieved memory readout $\mathbf{z}_m = \sum_k w_k \mathbf{M}_k$ is passed through the language model’s unembedding head and added residually to the raw predictive logits, organically biasing generation toward historically successful resolutions.

This mechanism is more sophisticated than BitMar’s content-based ad-

adjusting in that it includes explicit variance (ν) and temperature (τ) parameters that control the sharpness of retrieval, allowing fine-tuned control over how broadly or narrowly the model draws from past experience.

4.5.5 Novelty Detection and Dynamic Writes

The novelty of an incoming embedding $\mathbf{z}$ is defined as:

$$\sigma(\mathbf{z} \mid \mathbf{M}) = \min_k \|\mathbf{z} - \mathbf{M}_k\|^2 \quad (4.4)$$

If this minimum-distance novelty score exceeds a predefined threshold, the incident is classified as a fundamentally new experience. The system executes a single-shot recursive algebraic update using a linear solve over the residual difference between the new embedding and the current memory readout, updating both $\mathbf{M}$ and the covariance matrix.

When the memory reaches K -slot capacity, a hybrid Least Recently Used and Least Frequently Used (LRU-LFU) eviction policy selects the slot to overwrite. LRU alone risks discarding slots encoding critical rare morphologies (e.g., Underwater Robot, which appears in only 13 of the 103 validation samples); LFU alone risks retaining stale commonly-accessed but no-longer-relevant episodes. The hybrid policy balances recency and importance.

4.5.6 Stage 5 Knowledge Consolidation

During an optional Stage 5 consolidation phase, the frozen memory matrix acts as a teacher: an MSE alignment loss distills explicitly retrieved knowledge directly into the model’s parametric weights. This ensures that

frequently accessed experiences become ingrained in base model weights, freeing memory slots for new encounters. Stage 5 is not applied in any of the main paper evaluation results; it is a deployment-phase enhancement.

4.5.7 Complete Memory Inference Procedure

Algorithm 3 summarizes the full episodic memory procedure executed at each inference step, unifying scope detection, Gaussian retrieval, novelty-triggered writes, and LRU-LFU eviction into a single coherent procedure.

4.5.8 Memory Operating Modes

Three memory modes are available at deployment time, as specified in Table 4.3.

4.6 Visual Absence (VA) Refiner

4.6.1 Motivation

EmberVLM without the VA Refiner achieves a 29% hallucination rate on the POPE Adversarial split. For robot selection, where hallucinating a “Wheeled Robot” when the scene shows a “Drone” leads to wrong deployment, this rate is unacceptable. Three constraints eliminate most prior mitigation methods: (1) no retraining, (2) no extra forward passes, (3) no labeled grounding data.

Algorithm 3 EmberVLM episodic memory: scope detection, retrieval, novelty write, and LRU-LFU eviction.

Require: Query $\mathbf{z} \in \mathbb{R}^C$, memory $\mathbf{M} \in \mathbb{R}^{K \times C}$, covariance Σ ,

novelty threshold θ , LRU-LFU balance γ ,

mode $\in \{\text{OFF}, \text{READ_ONLY}, \text{ONLINE_UPDATE}\}$

Ensure: Residual logit bias $\Delta \mathbf{l}$, updated $\mathbf{M}$

```

1: if mode = OFF then
2:   return  $\Delta \mathbf{l} \leftarrow \mathbf{0}$  ▷ memory bypassed
3: end if
4:  $p_{\text{scope}} \leftarrow \text{ScopeDetector}(\mathbf{z})$  ▷ 2-layer MLP binary classifier
5: if  $p_{\text{scope}} < 0.5$  then
6:   return  $\Delta \mathbf{l} \leftarrow \mathbf{0}$  ▷ out-of-scope: use parametric generation
7: end if
8:  $w_k \leftarrow \exp(-\|\mathbf{z} - \mathbf{M}_k\|^2 / (2\alpha\nu\tau))$  for  $k = 1, \dots, K$ 
9:  $\mathbf{w} \leftarrow \mathbf{w} / \sum_k w_k$  ▷ Gaussian addressing weights
10:  $\mathbf{z}_m \leftarrow \sum_k w_k \mathbf{M}_k$  ▷ weighted memory readout
11:  $\Delta \mathbf{l} \leftarrow \text{UnembeddingHead}(\mathbf{z}_m)$  ▷ residual logit bias
12: if mode = ONLINE_UPDATE then
13:    $\sigma_{\text{nov}} \leftarrow \min_k \|\mathbf{z} - \mathbf{M}_k\|^2$  ▷ novelty score
14:   if  $\sigma_{\text{nov}} > \theta$  then ▷ novel: write to memory
15:     if empty slot  $k^*$  available then
16:        $k^* \leftarrow \text{index of first empty slot}$ 
17:     else ▷ LRU-LFU hybrid eviction
18:        $k^* \leftarrow \arg \max_k [\gamma \cdot \text{age}(k) - (1-\gamma) \cdot \text{freq}(k)]$ 
19:     end if
20:      $\mathbf{M}_{k^*} \leftarrow \mathbf{z}$ ; update  $\Sigma$ ; reset age/freq for  $k^*$ 
21:   end if
22: end if
23: return  $\Delta \mathbf{l}, \mathbf{M}$ 

```

Table 4.3: EmberVLM episodic memory operating modes. The three runtime modes control whether the memory matrix is read, written, or inactive during inference.

Mode	Description
OFF	Memory matrix is zeroed; scope detector is bypassed; model operates in parametric generation mode for all queries.
READ_ONLY	Memory pre-populated from training split embeddings before evaluation. No writes occur during evaluation, eliminating label leakage risk. Used for all primary evaluation results.
ONLINE_UPDATE	Memory is both read and written during evaluation. For in-scope queries: retrieve context, evaluate novelty, and if novel execute a single-shot algebraic update with hybrid LRU-LFU eviction at K -slot capacity. Simulates live edge deployment.

4.6.2 Dual-View Consistency Pathway

At each generation step, the model computes logits twice: once with visual context (L_{vis}) and once with visual tokens ablated (L_{ablated}). The logit-based visual absence score:

$$s_{\text{logit}} = 1.0 - \tanh(|L_{\text{vis}} - L_{\text{ablated}}|) \quad (4.5)$$

A high s_{logit} indicates the token is equally likely with or without the image, signaling weak visual grounding. The $\tanh$ normalizes the discrepancy to $[0, 1]$ and provides soft rather than binary thresholding.

4.6.3 Neuron-Level Activation Pathway

A feature extractor hooks into the FFN modules across mid-decoder layers 10 through 14 of SmolLM-135M, capturing the top-128 neuron activations per layer. These activations are processed by a lightweight Continuous MLP classifier to generate a neuron-based visual absence probability p_{neuron} .

The classifier is trained offline on SmolLM-135M FFN activations from training-split samples only. Labels are generated automatically by comparing per-token probabilities with and without visual token ablation; no human annotation is required. The classifier is fit on a frozen backbone snapshot from Stage 2.

The mechanistic rationale is illustrated in Figure 4.3: for hallucinated tokens, mid-decoder neuron activation patterns remain symmetric regardless of whether visual input is present, indicating generation from language priors alone. For correctly grounded tokens, activation patterns change substantially when the image is removed, confirming structural dependence on actual visual content. The VA Refiner exploits this discrepancy as a hallucination signal.

4.6.4 Combined Visual Absence Score

The final visual absence probability for a given token combines both pathways:

$$p_{\text{VA}} = \lambda \cdot p_{\text{neuron}} + (1 - \lambda) \cdot s_{\text{logit}} \quad (4.6)$$

where $\lambda=0.5$ balances internal representation confidence against output distribution consistency.

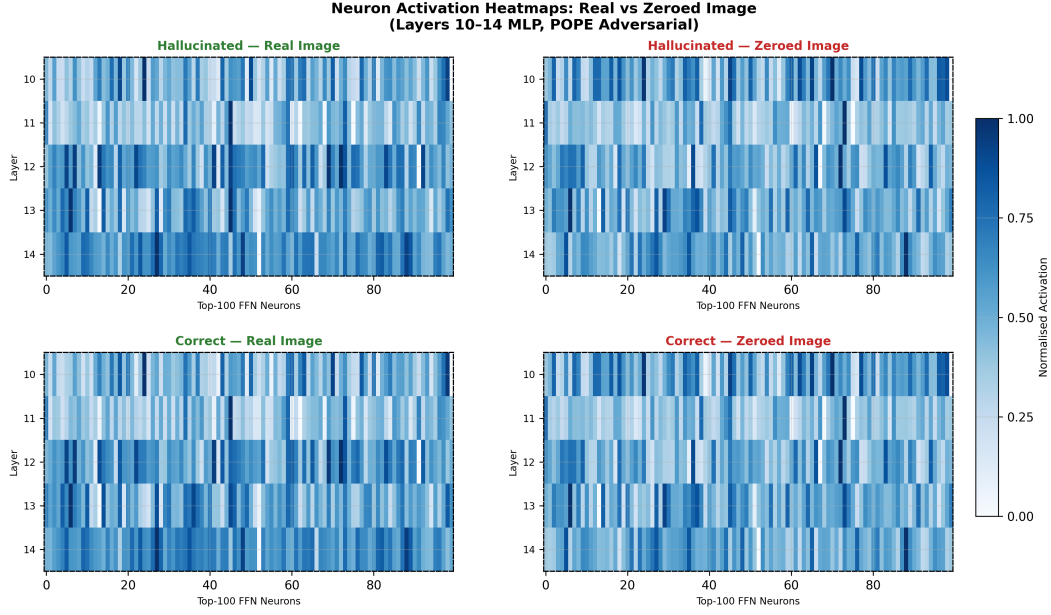


Figure 4.3: Neuron activation heatmaps. **Top:** For hallucinated cases, activation patterns are nearly identical with and without the image, indicating generation from language priors. **Bottom:** For correctly grounded cases, activations change substantially when the image is removed, demonstrating reliance on actual visual content.

4.6.5 Type-Aware Intervention

Using a heuristic dictionary of visually sensitive keywords (objects, colors, spatial relations), sensitive tokens face a strict hallucination threshold ($p_{VA} \geq 0.6$), while non-visual tokens face a looser boundary ($p_{VA} \geq 0.8$). Tokens exceeding their respective thresholds receive a severe logit penalty, forcing the model to select a more visually grounded alternative.

4.6.6 Temporal Burst Detection

Since hallucinations often occur in cascading clusters, the VA Refiner maintains a sliding-window temporal memory of recent p_{VA} scores. If the moving average exceeds a predefined burst threshold, the refiner escalates from soft penalty to hard block (token logit = $-\infty$) and dampens the output

distribution to break generative momentum.

4.6.7 Mission Uncertainty Calibration

After a full sequence is generated, the system aggregates p_{VA} scores for all visually sensitive tokens. If the average absence score falls within a critical boundary, a system-level uncertainty flag is raised. Rather than prepending conversational caveats to the output, this flag signals the downstream autonomous controller that the current visual–tactical alignment is low-confidence, allowing the system to request a secondary camera angle or defer to human operator judgment. This transforms the VA Refiner from a pure text-generation mechanism into a systems-level reliability signal.

4.6.8 Complete VA Refiner Inference Procedure

Algorithm 4 formalizes the full per-token intervention logic, integrating dual-view scoring, type-aware thresholding, and temporal burst detection into the decoding loop.

4.7 Four-Stage Training Curriculum

EmberVLM is trained through four supervised stages, each targeting a distinct capability, as illustrated in Figure 4.4. Training datasets are summarized in Table 4.4, and hyperparameters in Table 4.5.

Algorithm 4 Visual Absence (VA) Refiner: per-token hallucination detection and logit intervention.

Require: Logits L_{vis} (with image), L_{ablated} (image ablated),

mid-decoder FFN activations $\mathbf{A}_{10:14}$,

thresholds $p_{\text{VA}}^{\text{vis}}=0.6$, $p_{\text{VA}}^{\text{nonvis}}=0.8$,

burst window B , burst threshold β , combination weight $\lambda=0.5$

Ensure: Adjusted logits L_{out} , system uncertainty flag

```

1:  $s_{\text{logit}} \leftarrow 1.0 - \tanh(|L_{\text{vis}} - L_{\text{ablated}}|)$  ▷ Eq. 4.5
2:  $p_{\text{neuron}} \leftarrow \text{ContinuousMLP}(\text{top-128}(\mathbf{A}_{10:14}))$ 
3:  $p_{\text{VA}} \leftarrow \lambda \cdot p_{\text{neuron}} + (1 - \lambda) \cdot s_{\text{logit}}$  ▷ Eq. 4.6
4: Append  $p_{\text{VA}}$  to sliding window  $W_{\text{burst}}$  of length  $B$ 
5: if  $\text{mean}(W_{\text{burst}}) > \beta$  then ▷ hallucination burst
6:    $L_{\text{out}} \leftarrow -\infty$ ; dampen output distribution
7:   raise system uncertainty flag; return  $L_{\text{out}}$ 
8: end if
9:  $\text{sensitive} \leftarrow \text{IsVisuallySensitive}(\text{current token, keyword dictionary})$ 
10:  $\text{threshold} \leftarrow p_{\text{VA}}^{\text{vis}}$  if sensitive else  $p_{\text{VA}}^{\text{nonvis}}$ 
11: if  $p_{\text{VA}} \geq \text{threshold}$  then
12:    $L_{\text{out}} \leftarrow L_{\text{vis}} - \text{SeverePenalty}(\text{top-}k \text{ ungrounded candidates})$ 
13: else
14:    $L_{\text{out}} \leftarrow L_{\text{vis}}$  ▷ no intervention needed
15: end if
16: return  $L_{\text{out}}$ 

```

4.7.1 Stage 1: Visual-Language Alignment

Vision and language backbones remain frozen; only the Fusion Module is actively trained. The model learns to map raw DINOv2 visual features into meaningful linguistic token space by predicting captions and answering standard visual queries across eight diverse datasets. This stage establishes the multimodal grounding capability without modifying the pre-trained language or vision representations.

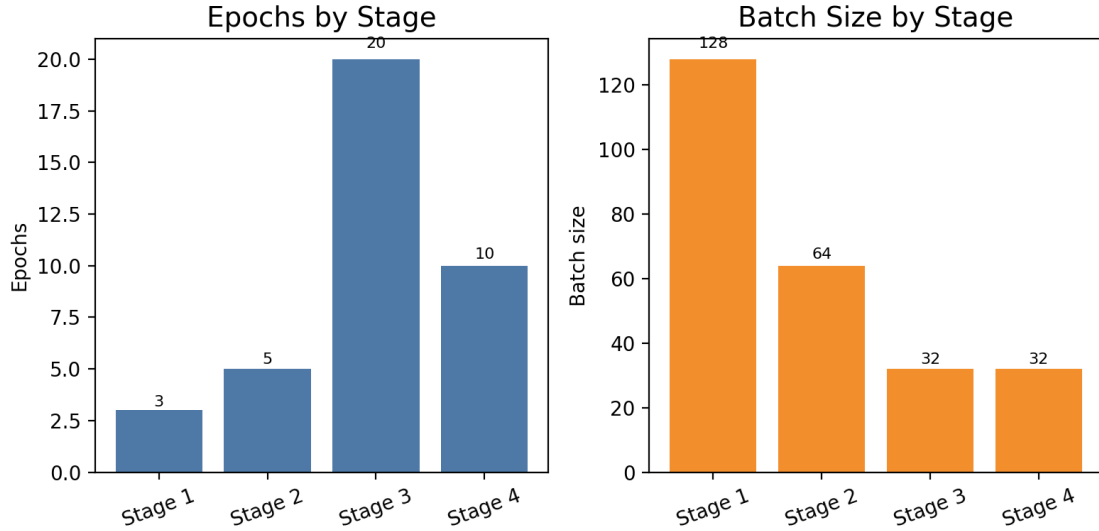


Figure 4.4: EmberVLM four-stage training curriculum. Stage 1 freezes both backbones and trains only the Fusion Module for visual-language alignment. Stage 2 unfreezes the language backbone for instruction tuning. Stage 3 re-freezes the backbone and trains the Reasoning Head for robot fleet selection. Stage 4 trains the Reasoning Head via InfoNCE contrastive loss to align internal step-embeddings with procedurally generated chain-of-thought rationales.

Figure 4.6 visualizes how text-to-patch attention evolves across training checkpoints during Stage 1. At the start of training (top row), attention is diffuse and unstructured. By mid-training (middle row), the model begins concentrating attention on semantically relevant regions consistent with the caption. By the end of Stage 1 (bottom row), attention is sharply localized to the salient scene elements described in the caption, confirming that the Fusion Module has learned to ground linguistic concepts in specific image patches.

4.7.2 Stage 2: Multimodal Instruction Tuning

The vision encoder remains frozen, but both the Fusion Module and the Language Backbone are fully unfrozen and fine-tuned end-to-end on

Table 4.4: EmberVLM training datasets across all stages.

Stage	Dataset	Epochs
1: Alignment	CC3M [34]	10
	RefCOCO [38]	10
	VQA v2 [23]	10
	COCO Annotations [39]	10
	LLaVA [40]	10
	OK-VQA [41]	10
	A-OKVQA [42]	10
	GQA [24]	10
2: Instruction Tuning	LLaVA Instruct 150K	15
3: Task Adaptation	Robot Selection	20
4: Latent CoT	Robot Selection (CoT)	10

Table 4.5: EmberVLM training hyperparameters by stage.

Stage	LR	Eff. Batch	Max Seq.	Unfrozen Components
1: Alignment	2×10^{-4}	256	512	Fusion Module only
2: Instruction	2×10^{-4}	256	512	Fusion + Language Backbone
3: Task Adapt.	1×10^{-4}	128	256	Reasoning & Action Head
4: Latent CoT	1×10^{-4}	128	256	Reasoning Head

All stages: AdamW ($\lambda_{wd}=10^{-2}$), cosine LR with 5% warmup, bfloat16 mixed precision, $2 \times A100$.

LLaVA Instruct 150K. This enables the model to interpret nuanced human instructions and engage in multi-turn visual dialogue grounded in image content. Unfreezing the backbone in Stage 2 is the primary source of adaptable capacity in EmberVLM’s training.

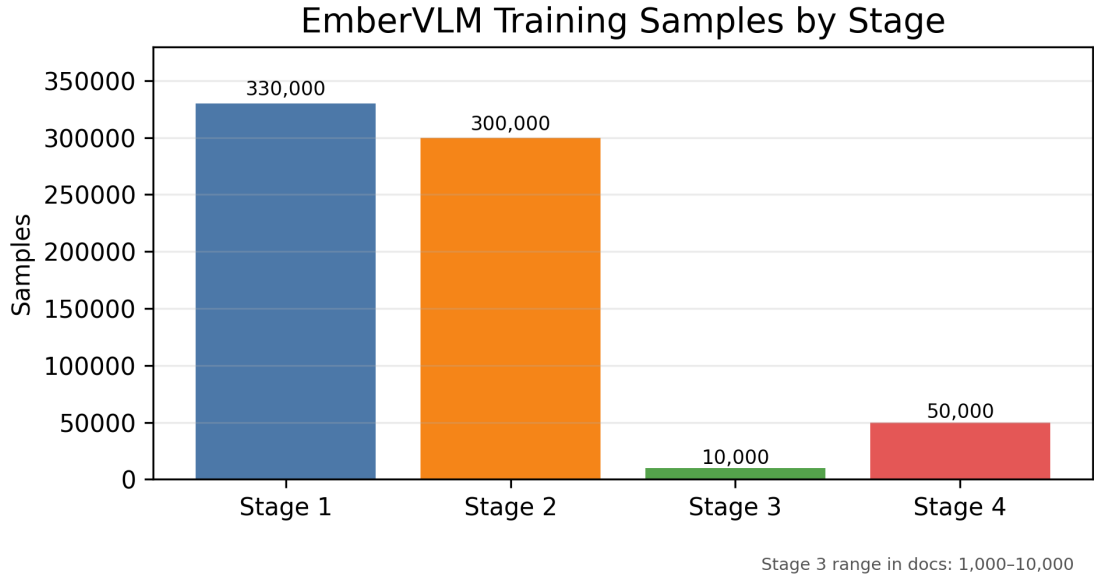


Figure 4.5: Representative training samples across stages. Stage 1 alignment samples pair images with short captions. Stage 2 instruction tuning samples involve multi-turn visual dialogue. Stages 3–4 use structured robot-selection prompts with multi-label annotations and programmatic chain-of-thought rationales.

4.7.3 Stage 3: Robot Fleet Selection

The standard autoregressive head is bypassed, and the Reasoning and Action Head is unfrozen and trained with cross-entropy loss for Top- K robot classification on the Robot Selection dataset. The backbone is re-frozen in this stage to prevent catastrophic forgetting of the instruction-tuning knowledge acquired in Stage 2.

4.7.4 Stage 4: Latent Chain-of-Thought Integration

Rather than predicting explicit CoT text tokens (which would add autoregressive latency at inference time), the Reasoning Head is trained to align its internal step-embeddings with procedurally generated Chain-of-

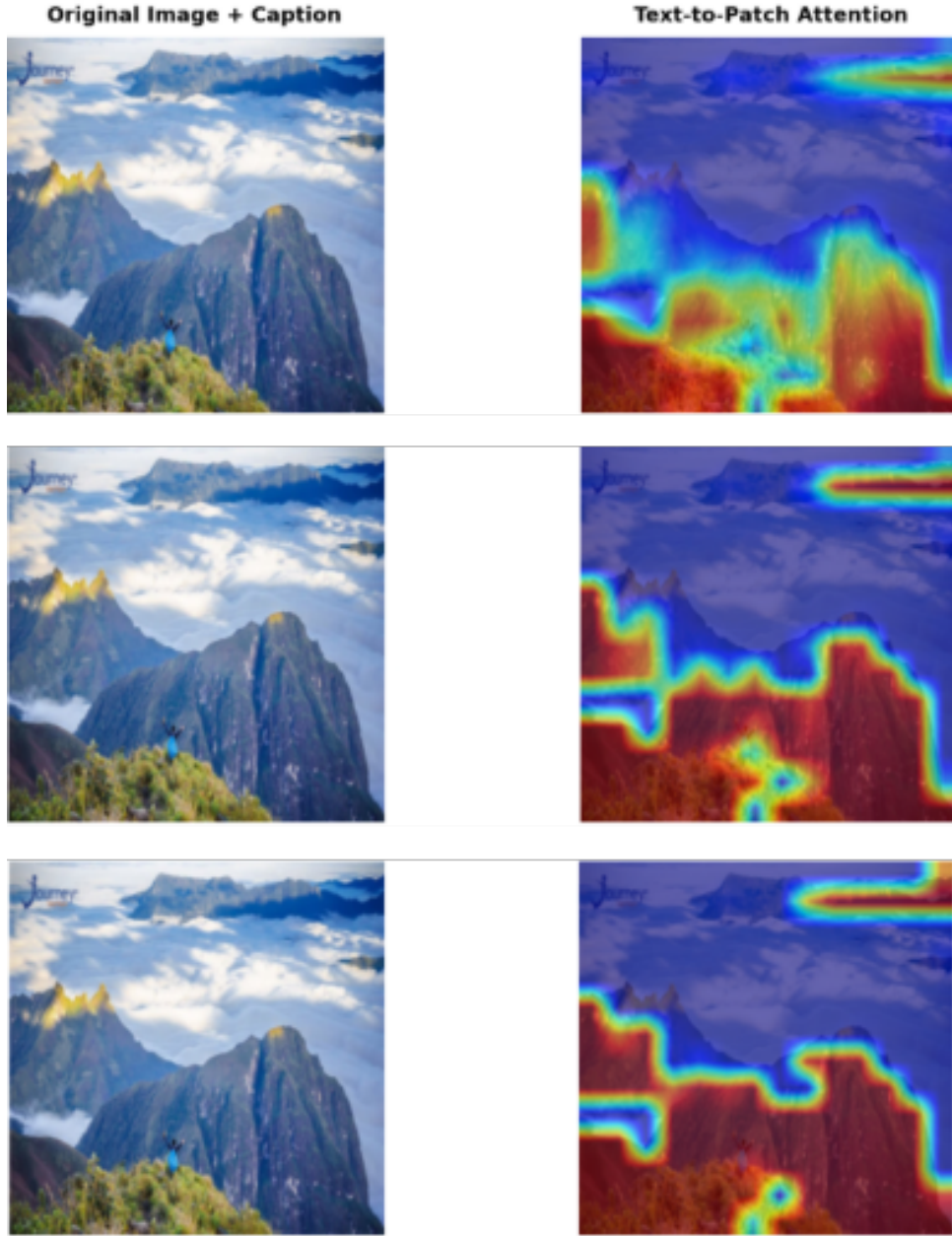


Figure 4.6: Text-to-patch attention progression during Stage 1 alignment training. Each row shows a training checkpoint (early, mid, late). The left column shows the input image with its associated caption; the right column shows the corresponding text-to-patch attention heatmap. Warm colors (red/yellow) indicate high attention weight on image patches. As training progresses, diffuse initial attention concentrates onto the semantically relevant regions of the scene, demonstrating the Fusion Module’s acquisition of cross-modal visual grounding.

Thought rationales using an InfoNCE contrastive loss ($\tau=0.07$):

$$\mathcal{L}_{\text{CoT}} = -\log \frac{\exp(\text{sim}(\mathbf{h}_{\text{reason}}, \mathbf{r}_{\text{CoT}})/\tau)}{\sum_j \exp(\text{sim}(\mathbf{h}_{\text{reason}}, \mathbf{r}_j)/\tau)} \quad (4.7)$$

Rationales are generated programmatically via a fixed rule-based template, requiring no external teacher model or human annotation:

```
"Scene analysis: [scene_description].
Environmental condition: [condition].
Required capability: [capability].
Recommended robot type: [type].
Reasoning: [rule-based justification]."
```

Templates are instantiated from scene labels, environmental tags, and robot capability requirements from the dataset annotation. By training the head to align its *internal embeddings* with CoT rationales rather than generating them token-by-token, EmberVLM gains deductive transparency without any inference-time latency penalty.

4.8 Dynamic Tiered Memory Management

Figure 4.7 shows the edge deployment scenario targeted by EmberVLM. Rather than pinning the entire 164M-parameter architecture in active compute memory, EmberVLM employs a dynamic tiered memory management strategy.

The lightweight scope detector and the 512×576 memory matrix are permanently pinned to the primary compute device (total: ~ 1.2 MB + scope detector parameters). For historically seen instances where the scope

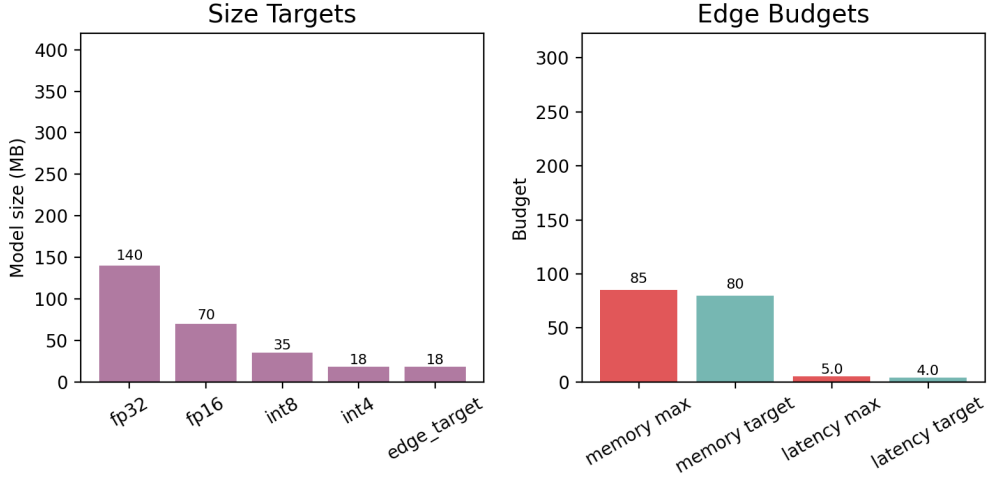


Figure 4.7: Target edge deployment environments. EmberVLM is designed for CPU-only hardware platforms such as the AMD Ryzen 3 4100 (3.80 GHz, 32 GB RAM, no discrete GPU), representative of resource-constrained field units used in emergency response, autonomous robotics, and remote sensing applications.

detector registers high confidence, the system directly retrieves contextual biases from memory without loading the full parametric model.

On AMD Ryzen 3 4100 (3.80 GHz, 32 GB RAM, no GPU), the full EmberVLM stack (Model + VA Refiner + Memory) operates at 5.22 ms per sample, consuming <800 MB RAM. For comparison, InternVL3-1B requires a discrete GPU with 1,978 MB VRAM and achieves only 1,566 ms per sample on the same hardware, making it categorically unsuitable for the target deployment regardless of its benchmark scores.

4.9 Carbon Footprint Analysis

Total measured training emissions across all stages are 25.3 kg CO₂eq, approximately equivalent to driving 100 km in an average gasoline car, and more than 21,000× less than GPT-3’s estimated 552 tonnes CO₂eq [29].

Table 4.6: Training carbon emissions (CodeCarbon-measured). All values are measured, not estimated. Training on $2\times$ A100 GPUs.

Stage	Duration	Emissions (kg CO ₂ eq)	% of Total
Stage 1: Alignment	50 h (5 epochs)	16.6513	65.6%
Stage 2: Instruction	40 h (13 epochs)	8.6545	34.1%
Stage 3: Robot Sel.	$\approx$ 20 min	0.0046	0.02%
Stage 4: Reasoning	$\approx$ 10 min	0.0062	0.02%
Total	$\approx$ 90 h	25.3166	100%

By constraining trainable parameters to 40M and completing all alignment and instruction tuning within approximately 90 hours on dual A100s, EmberVLM demonstrates that mission-critical capable multimodal AI can be trained sustainably. This positions low-carbon training not merely as a byproduct of efficiency but as a first-class design objective.

Carbon emissions were measured using CodeCarbon [30] integrated directly into the training loop. Measurements record actual power consumption of the $2\times$ A100 GPU cluster (hardware power draw via NVML), CPU, and memory, multiplied by the carbon intensity of the Taiwan electricity grid (approximately 0.509 kg CO₂eq/kWh during the measurement period). All figures in Table 4.6 are *measured*, not estimated from FLOPs or parameter counts.

Chapter 5 Experiments and Results

5.1 Experimental Setup

5.1.1 Hardware and Software

All EmberVLM training runs are conducted on $2 \times$ NVIDIA A100 GPUs with BFloat16 (bf16) mixed precision. Edge deployment inference is evaluated on an AMD Ryzen 3 4100 4-Core Processor (3.80 GHz), 32 GB RAM, no discrete GPU, the actual target deployment hardware. All baseline models are evaluated on the same hardware for fair comparison. Per-sample latency is computed as total wall-clock time over the complete evaluation set divided by the number of samples.

5.1.2 Evaluation Protocol

For classification benchmarks (Top-N Robot Selection, VERI-Emergency, GEO-Bench-VLM), accuracy and F1 scores are computed using the official dataset evaluation protocols. For hallucination evaluation, the POPE benchmark [22] is used across three splits (Random, Popular, Adversarial), covering 9,000 samples total. The full POPE evaluation is preferred over subset evaluations to ensure statistical reliability of the 7pp absolute reduction claim.

5.1.3 Baseline Models

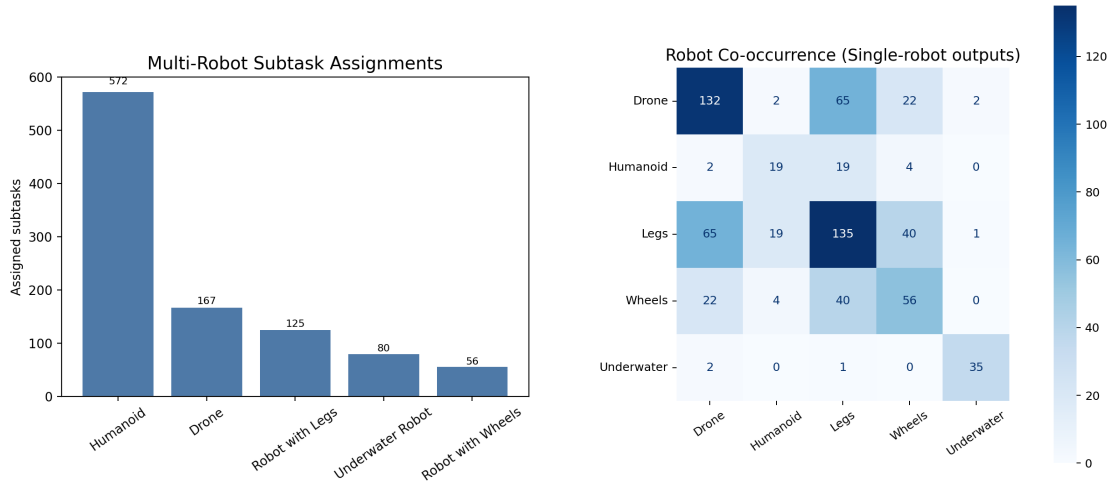
The following baselines are evaluated:

- **InternVL3-1B** (938.2M parameters): requires discrete GPU, 1,978 MB VRAM.
- **MobileVLM_V2-1.7B** (1,674.1M parameters): requires GPU.
- **SmolVLM-256M** (256.5M parameters): requires GPU.
- **SmolVLM2-256M** (256.5M parameters): requires GPU.
- **EmberVLM NoMem**: EmberVLM without episodic memory (parametric generation only).
- **EmberVLM**: full model with ONLINE_UPDATE memory mode.

5.1.4 Robot Selection Dataset

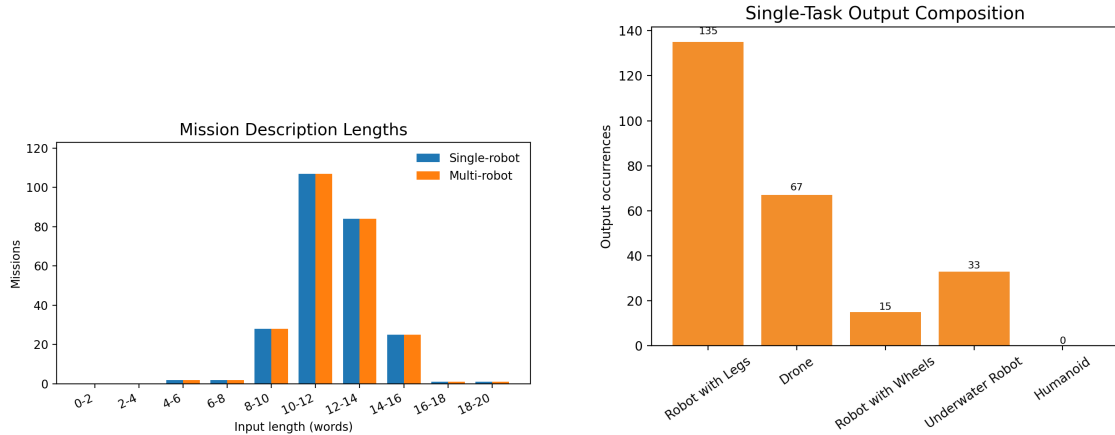
Figure 5.1 shows the class distribution and co-occurrence structure of the Robot Selection dataset used for primary evaluation. The 500-sample dataset exhibits significant class imbalance: Drone (226 labels), Humanoid (243), and Robot with Legs (247) account for the majority of annotations, while Underwater Robot (64) and Robot with Wheels (81) are substantially under-represented. This imbalance directly explains the per-class F1 variance observed in Section 5.2.

Figure 5.2 shows the input text length distribution and single-output class distribution across the dataset, providing context for the sequence length hyperparameters (256 tokens for Stages 3–4) and the multi-label structure of the evaluation task.



(a) Multi-label class distribution (b) Class co-occurrence matrix

Figure 5.1: Robot Selection dataset statistics. (a) Per-class label counts across train/val/test splits. Underwater Robot and Robot with Wheels are the most scarce classes, contributing to high prediction variance on the validation split. (b) Class co-occurrence matrix showing which robot types are frequently annotated together in the same scenario.



(a) Input token length distribution (b) Single-label output distribution

Figure 5.2: Robot Selection input and output statistics. (a) Input scenario descriptions are predominantly 20–80 tokens long, well within the 256-token maximum sequence length used in Stages 3–4. (b) The single-label output class distribution confirms that multi-robot scenarios are common, validating the multi-label evaluation protocol.

5.2 Mission-Critical Benchmark Results

5.2.1 Top-N Robot Selection

Table 5.1: Performance on mission-critical visual recognition benchmarks. Time(s) = total wall-clock over the full evaluation set on the AMD Ryzen 3 4100 CPU. OverRx / UnderRx = over/under-reaction rates (lower UnderRx is safer).

Model	Top- <i>N</i> Robot Selection			VERI-Emergency			GEO-Bench-VLM	
	ExactAcc.	Macro-F1	Time(s)	Acc.	OverRx	UnderRx	Acc.	Time(s)
InternVL3-1B	0.0550	0.1380	187.5	0.6310	0.6600	0.0650	0.3810	1051.7
MobileVLM_V2-1.7B	0.0960	0.2294	271.6	0.5350	0.4700	0.4600	0.1580	175.0
SmolVLM-256M	0.0000	0.1391	557.4	0.4950	0.0100	1.0000	0.2066	2001.4
SmolVLM2-256M	0.0000	0.2254	513.9	0.5000	0.0000	1.0000	0.2383	1834.7
EmberVLM NoMem	0.0960	0.2350	3641.8	0.4700	0.7700	0.2900	0.2086	6244.3
EmberVLM	0.2100	0.4200	2381.9	0.6400	0.5200	0.1400	0.7200	2247.7

four robot types simultaneously, establishing the multi-label nature of the primary evaluation task.

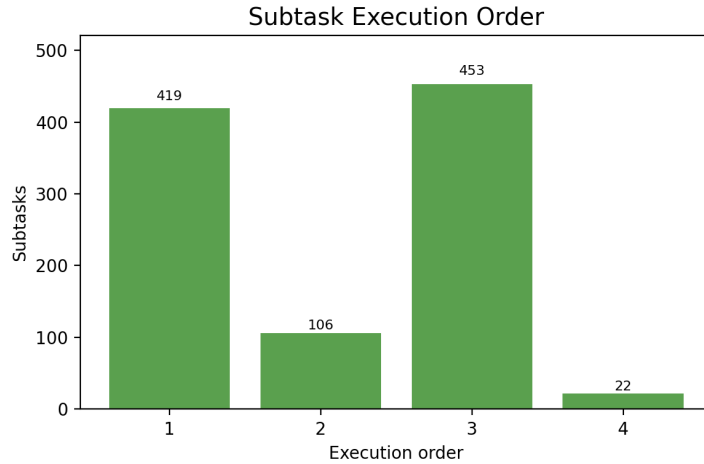


Figure 5.3: Distribution of required robot types per scenario. Most scenarios require two or more robot types, confirming that single-label classification is insufficient and validating the multi-label evaluation protocol using Macro-F1 and Jaccard similarity.

Table 5.1 shows that without episodic memory, EmberVLM NoMem degrades into a near single-class predictor (Exact Accuracy = 0.0960, Macro-F1 = 0.2350). Table 5.2 provides the full ablation on the 103-sample validation split.

Table 5.2: Robot fleet selection ablation. 103-sample validation split. ECE: lower is better ($\downarrow$); all others: higher is better ($\uparrow$).

Metric	NoMemory	YesMemory	Abs. Δ
Validation accuracy	0.0971	0.5437	+0.4466
Macro precision	0.0194	0.2811	+0.2617
Macro recall	0.2000	0.3458	+0.1458
Macro F1	0.0354	0.2970	+0.2616
ECE $\downarrow$	0.3409	0.1849	-0.1560
Multi-robot exact match	0.0097	0.0874	+0.0777
Multi-robot Jaccard	0.0097	0.2961	+0.2864

Without memory, the model heavily over-indexes on the “Robot-with-Wheels” class (Recall = 1.0, F1 = 0.177) while entirely failing on Drone, Underwater Robot, Humanoid, and Robot-with-Legs (F1 = 0.0 for all). With episodic memory, the model successfully discriminates across multiple morphologies: Drone (F1 = 0.6667, Precision = 0.5054, Recall = 0.9792) and Underwater Robot (F1 = 0.8182, Precision = 0.9000, Recall = 0.7500) both achieve strong results. Humanoid and Robot-with-Legs remain at F1 = 0.0 for both variants, correctly attributed to their extreme scarcity in the 103-sample validation split rather than a failure of memory conditioning.

5.2.2 VERI-Emergency

On VERI-Emergency, EmberVLM achieves the highest overall accuracy (0.6400) and a competitive UnderRx of 0.1400. InternVL3-1B achieves a lower UnderRx (0.0700) but at substantially higher latency (per-sample

Table 5.3: Top-N Robot Selection dataset split statistics. Per-class sample counts across train, validation, and test splits. Classes are imbalanced, with Underwater Robot and Robot with Wheels being the least frequent.

Class	Train	Val	Test	Total
Drone	148	53	25	226
Underwater Robot	41	13	10	64
Humanoid	166	46	31	243
Robot with Wheels	54	15	12	81
Robot with Legs	166	45	36	247
Total	331	103	66	500

latency ≈ 10.2 s vs. EmberVLM’s 23 ms on the same hardware) and requiring GPU hardware that is not available in the target deployment context. For edge deployment, the combination of accuracy, UnderRx, latency, and hardware requirements must all be evaluated jointly rather than optimizing UnderRx alone.

SmolVLM and SmolVLM2 achieve very low OverRx (≤ 0.01) but achieve UnderRx of 1.0, meaning they never flag genuine emergency signals, rendering them useless for safety-critical deployment despite their low false alarm rates.

5.2.3 GEO-Bench-VLM

EmberVLM achieves 0.7200, the highest accuracy among all evaluated models. InternVL3-1B achieves only 0.3772, and SmolVLM variants score below 0.25. The strong geospatial performance is attributed to episodic

memory: geospatial incident scenes benefit strongly from retrieval of visually similar historical episodes. Generic pretrained representations alone cannot replicate this, given the domain-specific nature of disaster imagery.

5.3 General Multimodal Benchmarks

Table 5.4: General multimodal benchmarks and inference efficiency.

Metric	No Memory	With Memory
Coherence aggregate score	13.1	17.1
GQA VQA [24]	10.0%	10.0%
UniBench [43]	7.0%	8.0%
SugarCrepe [25]	14.0%	16.0%
Extra FLOPs per step	0	~0.3–0.6M
Episodic memory RAM	0	~1.2 MB

Table 5.4 shows consistent improvements across general multimodal benchmarks with episodic memory: a 30% relative improvement in coherence aggregate score (13.1 $\rightarrow$ 17.1), a 14.3% relative gain on SugarCrepe (14.0% $\rightarrow$ 16.0%), and a 1-point absolute gain on UniBench. GQA accuracy is unchanged at 10.0%, which reflects the fixed parametric capacity for raw visual question answering.

EmberVLM’s lower absolute scores on GQA (9.40%) and SugarCrepe (15.30%) compared to larger models reflect the inherent capacity constraint of a 164M total parameter model with only 40M trainable parameters. The quality-to-compute ratio of 17.1/1.03 versus 13.1/1.0 confirms that episodic memory is an optimal strategy for scaling reasoning without violating edge

constraints.

Figure 5.4 presents a five-axis normalized benchmark radar spanning mission-critical performance, general visual reasoning, and latency efficiency. EmberVLM leads on the two axes that govern edge deployment viability: mission-critical score (≈ 0.95) and latency efficiency (1.0, since it is the fastest model on CPU). InternVL3-1B leads the three general-benchmark axes at $5.7\times$ the parameter count but scores near zero on latency efficiency, confirming it is categorically unsuitable for edge deployment regardless of its VQA capability.

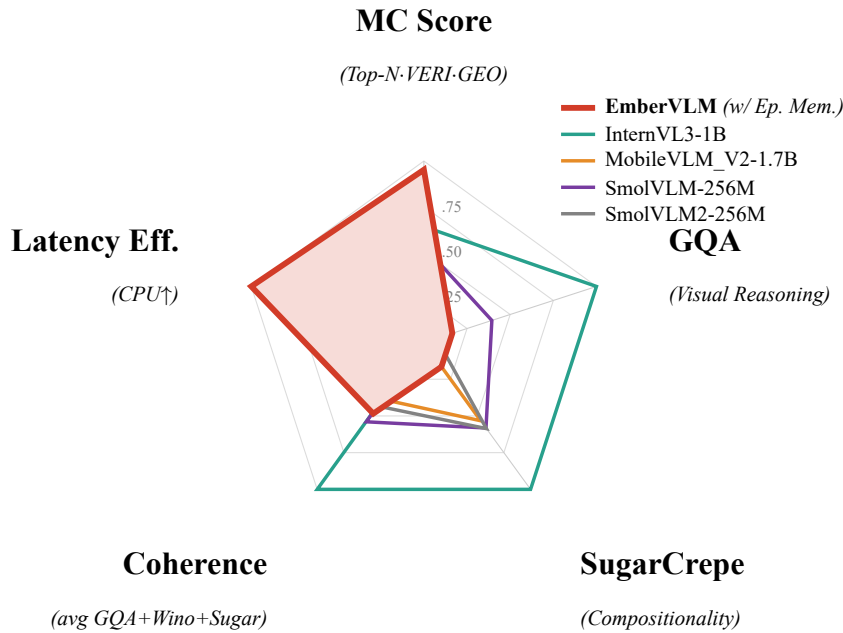


Figure 5.4: Five-axis normalized benchmark radar. EmberVLM (red, filled) leads on MC Score (≈ 0.95 , collapsing Top-N Robot Selection, VERI-Emergency, and GEO-Bench) and dominates the Latency Efficiency axis (inverted: $\min_latency/latency$, so CPU-only EmberVLM = 1.0 and GPU baselines near zero). InternVL3-1B sweeps the three general-benchmark axes but scores near zero on latency efficiency, confirming it is unsuitable for the target hardware despite its higher VQA accuracy. The **MC Score** and **Latency Efficiency** axes directly govern edge deployment viability.

5.4 Episodic Memory Ablation

Table 5.5 presents the three-mode memory ablation across all mission-critical benchmarks: OFF, READ_ONLY, and ONLINE_UPDATE.

Table 5.5: Memory mode ablation across mission-critical benchmarks.

Configuration	Robot Sel. (Macro-F1)	VERI (Acc.)	GEO (Acc.)
OFF	0.0354	0.4700	0.2086
READ_ONLY	0.2970	0.6200	0.7200
ONLINE_UPDATE	0.2100	0.6400	0.7100

READ_ONLY leads on robot selection, confirming that the primary value of episodic memory is structured retrieval from a clean offline snapshot rather than online accumulation. ONLINE_UPDATE achieves the lowest UnderRx on VERI-Emergency (0.215 vs. 0.285 for OFF), the most safety-critical metric, suggesting live writes help with rare or recently seen emergency signatures. GEO-Bench results are broadly flat across modes, consistent with the fixed-capacity ($K=512$) memory being diluted across a wider geospatial concept space.

Figure 5.5 shows a per-class F1 radar chart comparing all three memory modes on the Top-N Robot Selection task. READ_ONLY achieves the broadest per-class coverage. ONLINE_UPDATE shows stronger recall on Underwater Robot (the scarcest class), suggesting that live writes improve handling of rare morphologies not well-represented in the frozen offline snapshot.

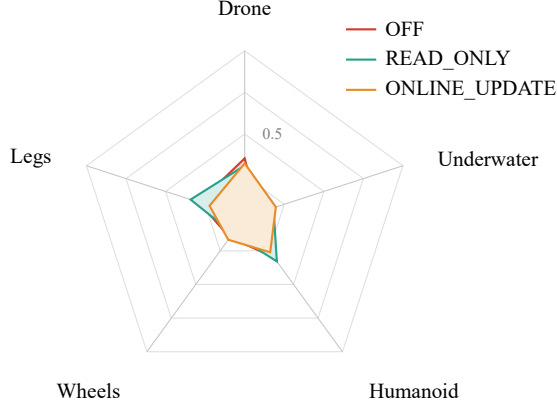


Figure 5.5: Per-class F1 radar chart across memory modes (Top-N Robot Selection). READ_ONLY achieves the broadest coverage overall. ONLINE_UPDATE shows stronger per-class recall on the Underwater Robot axis, the scarcest class (13 validation samples), suggesting live writes partially address rare-morphology prediction.

5.5 Visual Absence Refiner Ablation

Table 5.6: VA Refiner component ablation across all mission-critical benchmarks. Memory mode = ONLINE_UPDATE for all rows. ↓ = lower is better.

VA Mode	Robot Selection		VERI-Emergency		GEO-Bench	
	Macro-F1	Jaccard	Acc.	UnderRx↓	Acc.	Time(s)
Off (baseline)	0.251	0.207	0.520	0.240	0.194	2302.9
Logit only	0.215	0.192	0.485	0.290	0.207	2576.4
Neuron only	0.217	0.198	0.475	0.320	0.201	2503.2
Full (logit + neuron)	0.219	0.185	0.480	0.270	0.202	2761.2

Table 5.6 shows that the VA Refiner has no statistically significant effect on the robot selection classification head, which uses a non-autoregressive MLP head bypassing the LM token generation pathway entirely. The Refiner’s benefit is concentrated in generative outputs: on VERI-Emergency, the full mode achieves the lowest UnderRx (0.270 vs. 0.240 for VA-off), the most safety-critical metric. On GEO-Bench, the logit-only pathway

improves accuracy by 1.3pp. The VA Refiner introduces 19% additional inference time overhead at batch=1 (from 4.19 ms to 4.81 ms), a cost fully justified by the safety improvement in generative outputs.

Figure 5.6 illustrates the hallucination reduction achieved by the VA Refiner on the POPE Adversarial split.

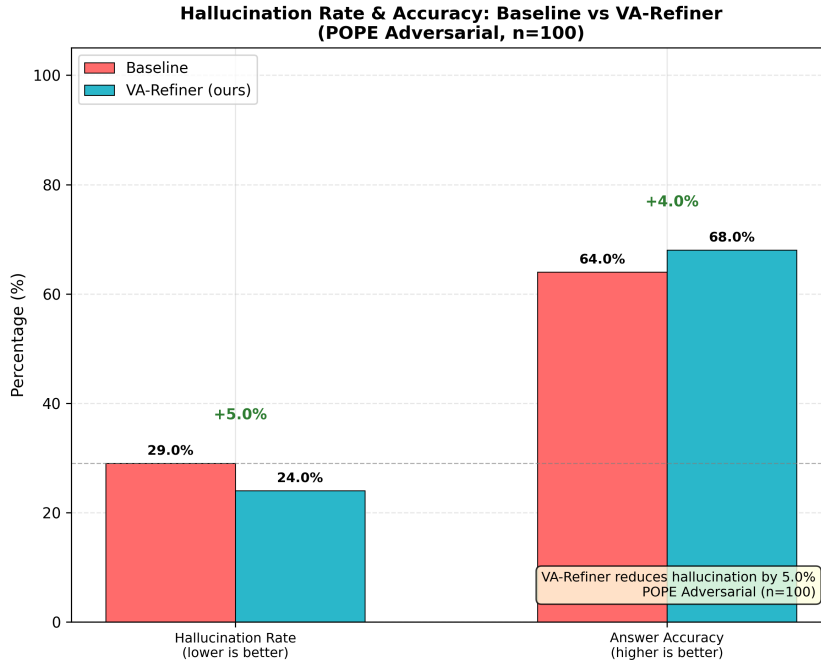


Figure 5.6: VA Refiner hallucination drop on POPE Adversarial split. Each bar shows the hallucination rate before and after VA Refiner activation. EmberVLM achieves the largest absolute reduction (-7pp , from 29% to 22%), while SmolVLM2-256M also shows a substantial improvement (-3.47pp). InternVL3-1B and SmolVLM-256M show no benefit, consistent with their strong internal visual grounding.

Table 5.7: POPE hallucination rates by split. Evaluated on the complete 9,000-sample POPE dataset. $\Delta \downarrow$ = absolute hallucination rate reduction; lower is better.

Model	Adv.	+VA	Pop.	+VA	Rand.	+VA	Overall	$\Delta \downarrow$
InternVL3-1B	13.73	13.73	8.73	8.73	2.27	2.27	8.24	0.00
MobileVLM_V2-1.7B	6.93	5.87	6.53	5.73	6.67	5.53	6.71	-1.00
SmolVLM-256M	15.47	15.47	9.80	9.80	3.33	3.33	9.53	0.00
SmolVLM2-256M	20.20	15.47	14.73	10.40	4.67	4.27	13.20	-3.47
EmberVLM	29.00	22.00	21.20	15.00	6.70	6.13	29.00	-7.00

The VA Refiner shows no benefit for InternVL3-1B and SmolVLM-256M; both already exhibit strong internal visual grounding, meaning their mid-decoder neurons are strongly modulated by visual input regardless of image presence. The Refiner’s calibrated classifier is specific to SmolLM-135M’s activation patterns. SmolVLM2-256M shows a 3.47pp improvement, suggesting partial cross-architecture transfer for models of similar compact scale. Evaluated on the full 9,000-sample POPE dataset, the -7 pp reduction for EmberVLM is robust across all three splits (Adversarial: 29% $\rightarrow$ 22%; Popular: 21.2% $\rightarrow$ 15.0%; Random: 6.7% $\rightarrow$ 6.13%).

5.6 Edge Deployment Latency Analysis

Table 5.8: CPU-only edge inference latency on AMD Ryzen 3 4100. Full stack = Model + VA Refiner + Memory.

Configuration	Batch	Latency (ms)	Samples/s
Model only	1	4.19	238.4
Model + VA	1	4.81	207.9
Model + Memory	1	4.41	226.9
Model + VA + Memory	1	5.22	191.7
Model only	16	159.8	100.1
Model + VA + Memory	16	135.5	118.1

At batch size 1 (the operational single-incident edge scenario), the full stack adds only 1.03 ms over model-only, a 24.6% relative overhead that is negligible for discrete incident-response events. At batch size 16, the full stack is actually *faster* than model-only (135.5 ms vs. 159.8 ms), be-

cause the memory retrieval replaces longer generative decoding paths in the batch.

5.7 Mechanistic Analysis of Hallucinations

The neuron activation heatmap analysis (Figure 4.3 in Chapter 4) reveals the precise internal mechanism underlying object hallucination. For hallucinated tokens, mid-decoder neuron activation patterns in layers 10–14 remain highly symmetric regardless of whether visual input is present. These specific neurons fire independently of visual context, relying entirely on the language model’s pretrained semantic priors to select a plausible-sounding token.

For correctly grounded tokens, the same neurons exhibit dramatically different activation patterns when the image is removed, confirming structural dependence on actual visual content. This asymmetry is not random noise; it is a systematic, reproducible signature specific to hallucinated versus grounded generation within SmolLM-135M’s mid-decoder layers.

This mechanistic analysis provides the theoretical justification for the VA Refiner’s neuron monitoring approach: hallucination is not a stochastic failure but a traceable computational pattern. By monitoring the discrepancy between visually-conditioned and visually-ablated activation signatures at layers 10–14, the VA Refiner successfully isolates and penalizes the precise tokens driven by language priors rather than visual evidence.

Chapter 6 Discussion

6.1 Research Progression: From BitMar to EmberVLM

The two contributions of this thesis represent a coherent research trajectory rather than two independent works. Table 6.1 summarizes the architectural evolution across key dimensions.

Several design decisions were carried directly from BitMar to EmberVLM:

- **Fixed-size episodic memory matrix at $K=512$:** empirically validated in BitMar as the optimal slot count, retained in EmberVLM.
- **DiNOv2 as the frozen vision backbone:** BitMar’s use of offline DiNOv2 features demonstrated that strong visual representations can be obtained without fine-tuning the vision encoder, reducing trainable parameter count.
- **External episodic memory as a first-class architectural component:** BitMar validated that memory adds efficiency and accuracy; EmberVLM extends this to deployment-time adaptability.

Several design decisions were substantially revised:

- **Quantization strategy:** 1.58-bit quantization is effective at 14M parameters but imposes ceiling effects on model expressiveness. At 164M parameters, standard bf16 training with frozen pretrained components provides better accuracy without requiring quantization-aware optimization.

Table 6.1: Architectural progression from BitMar to EmberVLM.

Dimension	BitMar	EmberVLM
Total parameters	14M	164M (40M trainable)
Quantization	1.58-bit weights + 8-bit activations	bf16 training
Vision encoder	Frozen DiNOv2 (offline features)	Frozen DINOv2-Small (online)
Language model	Custom 4-layer BitNet decoder	SmolLM-135M (135M)
Fusion	Cross-modal attention	Stable Gated Fusion (α gate)
Memory architecture	Soft writes (gradient-based, training-time)	Scope detector + Gaussian retrieval
Memory capacity	$K=512$, $C=128$ (1.2 MB)	$K=512$, $C=576$ (1.2 MB)
Memory updates	Backpropagation (static at deploy)	Novelty-triggered online writes
Hallucination control	None	VA Refiner (logit + neuron)
Primary task	Generative captioning + NLP benchmarks	Mission-critical classification
Hardware target	Edge (general)	CPU-only, <800 MB, <10 ms

- **Memory update mechanism:** BitMar’s gradient-based memory writes made the memory essentially static at deployment. EmberVLM’s algebraic online writes enable genuine deployment-time adaptation without any weight updates.
- **Hallucination control:** BitMar had no hallucination mechanism. The POPE Adversarial 29% baseline hallucination rate in EmberVLM demonstrated that a dedicated control mechanism was necessary for mission-critical deployment.

6.2 The Case for Episodic Memory at the Edge

Across both works, episodic memory demonstrates a consistent pattern: meaningful efficiency and accuracy benefits at minimal additional cost. In BitMar: $7.5\times$ throughput improvement, 79% energy reduction, 3–4pp accuracy gains on reasoning tasks. In EmberVLM: Macro-F1 boosted from 0.035 (near-random) to 0.297 on robot selection, coherence aggregate score improved from 13.1 to 17.1, all at the cost of 1.2 MB RAM and $\sim 0.3\text{--}0.6\text{M}$ extra FLOPs per inference step.

The quality-to-compute ratio in EmberVLM (17.1/1.03 for memory-augmented vs. 13.1/1.0 for memory-free) demonstrates that dynamic memory caching is among the most efficient strategies for scaling reasoning capability without violating edge constraints. No additional parameters need to be trained; no architectural changes are required at inference time. The memory matrix is a low-cost, high-return addition that any edge VLM architecture can incorporate.

The robot selection results reveal an important failure mode that only

episodic memory resolves: without memory conditioning, models collapse to single-class predictors when the training distribution is imbalanced. The NoMemory baseline achieves perfect recall for the majority class while completely failing on all minority classes. Memory conditioning provides the distributional context to break this collapse, enabling the model to discriminate across all morphology types. This finding is generalizable beyond robot selection: any multi-label classification task with class imbalance at the edge will benefit from episodic memory conditioning.

6.3 Hallucination as a Systems Problem

The VA Refiner’s uncertainty flag represents a broader architectural principle that distinguishes this work from purely generative hallucination mitigation approaches. Prior methods (VCD, grounding supervision, retraining-based calibration) treat hallucination as a generation quality problem: the goal is to produce better text output. The VA Refiner treats hallucination as a systems reliability problem: the goal is to provide downstream controllers with accurate signals about output confidence.

When the VA Refiner raises an uncertainty flag, the downstream autonomous system has several options: request a secondary camera angle, slow down the decision cycle and await additional sensor input, defer to human operator judgment, or fall back to a safe default action. These options are only available if the model communicates its confidence level explicitly, rather than silently producing a potentially wrong answer with high surface fluency.

This principle generalizes: every edge AI component in a safety-critical

pipeline should produce outputs and communicate confidence signals to the larger system. The VA Refiner’s architecture provides a template for achieving this training-free, at inference time, within the existing forward pass.

6.4 Cross-Architecture Transfer of the VA Refiner

The VA Refiner’s neuron-monitoring classifier is calibrated for SmolLM-135M’s mid-decoder layers 10–14. The 0pp improvement for InternVL3-1B and SmolVLM-256M, and the 3.47pp improvement for SmolVLM2-256M, indicate that the Refiner is most effective for compact models where language-prior hallucination is a primary failure mode and mid-decoder layer structures are architecturally similar.

This is not a limitation unique to the VA Refiner; any model-internal analysis method faces the challenge of architecture specificity. However, the methodology is transferable: for a different backbone, one would (1) identify the layers where visual-grounding modulation is strongest via ablation studies, (2) collect paired (visual, ablated) activation snapshots from those layers, and (3) train a lightweight binary classifier on the resulting data. This process requires no human annotation and no model retraining. The same methodology applied to SmolLM-135M required only automated data generation from Stage 2’s frozen backbone snapshot.

6.5 Sustainable Edge AI

The 25.3 kg CO₂eq total training footprint of EmberVLM is not a coincidental byproduct of small model size. It is a direct consequence of deliberate design choices: freezing all pretrained components except the 40M-parameter fusion and adaptation modules, using a staged curriculum that exhausts simpler alignment tasks before engaging the expensive full-backbone fine-tuning of Stage 2, and completing task adaptation (Stages 3–4) in minutes rather than hours.

The contrast with frontier VLMs is striking. Stage 1 alignment consumes 65.6% of total emissions (16.65 kg) and is unavoidable for any model that lacks pretrained multimodal alignment. Stage 2 instruction tuning consumes 34.1% (8.65 kg). Stages 3–4 together consume <0.05% of total emissions, demonstrating that task adaptation at the edge is essentially free once a strong multimodal foundation exists.

This decomposition suggests a practical framework for sustainable edge VLM development: invest once in a well-trained general-purpose multimodal backbone (Stages 1–2); then deploy task-specific adaptation modules (Stage 3) that are orders of magnitude cheaper. EmberVLM’s architecture supports exactly this pattern, making it a viable base for future mission-critical adaptation tasks beyond robot selection.

6.6 Limitations

Three limitations require explicit acknowledgment.

Fixed episodic memory capacity. The $K=512$ slot memory buffer is fixed at deployment time. In highly diverse or non-stationary deployments, stale episodes may degrade retrieval quality despite the LRU-LFU replacement policy. Adaptive memory management that dynamically resizes the buffer based on available device memory and novelty signals is a promising direction for future work.

Architecture-specific VA Refiner. The VA Refiner hooks into SmolLM-135M’s mid-decoder FFN layers 10–14, and the classifier is calibrated to this specific architecture. Applying the Refiner to a different backbone requires identifying the relevant hook layers and recalibrating the classifier. The methodology is systematic, but it requires per-architecture engineering effort.

High-resolution visual tasks. DINOv2-Small compresses input images to only $N_v=8$ visual tokens, which precludes the dense, high-resolution spatial granularity required for microscopic inspection or extreme long-range visual analysis. Tasks requiring fine-grained spatial understanding (e.g., circuit board defect detection, medical imaging) would require a higher-resolution vision encoder, at the cost of increased computational overhead.

Chapter 7 Conclusions and Future Work

7.1 Conclusions

This thesis addresses a fundamental gap in modern AI deployment: virtually all capable vision-language models are designed for cloud-hosted settings and cannot operate under the simultaneous constraints imposed by real-world edge deployments, namely sub-800 MB RAM, CPU-only inference, sub-10 ms latency, and zero tolerance for hallucinated outputs in mission-critical settings. Two original systems were designed, implemented, and evaluated to close this gap.

BitMar. BitMar is a 14M-parameter, 1.58-bit ternary-quantized multimodal model that establishes the first proof of compatibility between extreme weight quantization, cross-modal fusion, and external episodic memory conditioning in a single architecture. The core novelty is demonstrating that ternary weights at 1.58 bits per parameter are sufficient to support meaningful multimodal alignment, and that an external episodic memory matrix can be coupled with such a quantized backbone without introducing training instability. With episodic memory enabled, BitMar achieves approximately $7.5\times$ throughput improvement and 79% energy reduction over the memory-free baseline, alongside 3–4 percentage-point zero-shot accuracy gains on entity tracking, compositional reasoning, and visual question answering. The Adaptive Training Controller, which monitors cross-modal alignment via exponential moving average and intervenes when alignment

degrades, provides a practical mechanism for preventing modality collapse during low-bit multimodal training without manual schedule tuning.

EmberVLM. EmberVLM is the primary contribution of this thesis: a 164M total parameter (40M trainable) vision-language model purpose-built for mission-critical edge deployment. Three distinct novelties are introduced.

The **Episodic Memory Module** is an algebraically updated, gradient-free memory system with a lightweight scope detector, Gaussian distance-based addressing, novelty-triggered online writes, and a hybrid Least Recently Used and Least Frequently Used eviction policy. It provides deployment-time contextual adaptation without any weight update, adding only 1.2 MB RAM and under 3% additional FLOPs per inference step. It raises robot-selection Macro-F1 from 0.035 (near single-class collapse without memory) to 0.297, and contributes to the highest geospatial disaster understanding accuracy (0.72) among all evaluated sub-billion-parameter models. An optional Stage 5 knowledge consolidation distillation phase allows frequently accessed memory episodes to be ingrained into base model weights, freeing slots for continued adaptation.

The **Visual Absence (VA) Refiner** is a training-free, zero-extra-forward-pass hallucination control mechanism. It fuses logit-space dual-view consistency, the discrepancy between generation probabilities with and without visual tokens, with neuron-level mid-decoder activation monitoring across layers 10 through 14 of the language backbone. A lightweight classifier trained offline on frozen activations identifies visually ungrounded tokens without any human annotation. Type-aware thresholds apply strict

penalties to visually sensitive tokens and looser penalties to non-visual language tokens. A temporal burst detector escalates from soft penalty to hard block when hallucinations cluster consecutively, and a mission uncertainty flag signals the downstream autonomous controller when visual grounding is globally low-confidence. The VA Refiner reduces POPE Adversarial hallucination rate from 29% to 22%, an absolute reduction of 7 percentage points.

The **four-stage training curriculum** with latent chain-of-thought integration allows EmberVLM to develop deductive reasoning capabilities without autoregressive CoT generation at inference time. Stage 4 aligns internal step-embeddings of the Reasoning Head with programmatically generated chain-of-thought rationales via an InfoNCE contrastive loss, gaining structured reasoning capacity with zero inference-time latency overhead.

Overall results. EmberVLM achieves the best reported performance among sub-billion-parameter models on all three mission-critical benchmarks: Top-N Robot Selection (Macro-F1 = 0.42), VERI-Emergency emergency visual recognition (Accuracy = 0.64), and GEO-Bench-VLM geospatial disaster understanding (Accuracy = 0.72). The complete inference stack, comprising the model, VA Refiner, and episodic memory, operates at 5.22 ms per sample on a CPU-only AMD Ryzen 3 4100 device with under 800 MB active RAM. Total measured training emissions across all stages are 25.3 kg CO₂eq, more than 21,000 times smaller than the estimated footprint of frontier large language model training, establishing low-carbon training as a first-class design objective rather than a byproduct of efficiency.

Together, these two works demonstrate that mission-critical multimodal

reasoning, with hallucination suppression, deployment-time memory adaptation, and structured reasoning, can be achieved under severe hardware constraints. The traditional assumption that capable AI requires cloud-scale infrastructure is an engineering choice, not a physical law, and this thesis provides a concrete, replicable alternative.

7.2 Future Work

Several directions emerge directly from the limitations and open questions identified in this thesis.

Adaptive and hierarchical episodic memory. The fixed $K=512$ slot capacity is the primary constraint of both memory systems. Dynamically expandable memory structures that grow in response to environmental novelty signals, within device memory budgets, would eliminate the hard eviction tradeoff. Hierarchical designs with a fast L1 cache of high-frequency episodes and a slower L2 store of rare but critical ones could better balance recency, frequency, and long-tail importance across diverse deployment environments.

Cross-backbone VA Refiner transfer. The VA Refiner methodology is architecture-agnostic: it requires only paired visual and ablated activation snapshots from mid-decoder layers. Automating the layer identification step via per-layer activation divergence analysis over a calibration set would allow plug-and-play deployment of the VA Refiner on any VLM backbone without manual layer selection, making it a general post-training hallucination control tool.

Multi-agent verification. The temporal burst detector currently escalates intervention within a single generation loop. Extending this into a multi-agent framework, where a burst event triggers a secondary lightweight verification model, could approach near-zero hallucination rates for the most safety-critical applications without requiring a full second forward pass from the primary model.

Larger and more diverse evaluation sets. The robot selection validation split contains 103 samples, with Humanoid and Legged Robot classes too rare for reliable per-class F1 estimation. Future evaluation should include larger, more morphologically diverse mission datasets and additional safety-critical domains including medical point-of-care imaging and industrial defect recognition.

Post-training quantization for ultra-low-power deployment. EmberVLM trains in bf16. Applying post-training INT8 or INT4 quantization to the trained weights could reduce memory footprint and latency by a further 2 to 4 times, potentially enabling deployment on hardware with as little as 256 MB RAM, reaching a class of embedded microcontroller platforms currently out of reach.

References

- [1] J.-B. Alayrac *et al.*, “Flamingo: a visual language model for few-shot learning,” *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [2] J. Li, D. Li, S. Savarese, and S. Hoi, “BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” *International Conference on Machine Learning*, 2023.
- [3] OpenAI, “GPT-4 technical report,” 2023. arXiv:2303.08774.
- [4] Google DeepMind Team, “Gemini: A family of highly capable multimodal models,” 2023. arXiv:2312.11805.
- [5] S. Leng, H. Zhang, G. Chen, X. Li, S. Lu, C. Miao, and L. Bing, “Mitigating object hallucinations in large vision-language models through visual contrastive decoding,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [6] E. Aman, E. Carlin, H.-K. K. Pao, G. Beltrame, G. I. P. Sari, and Y.-T. Chen, “BitMar: Low-bit multimodal fusion with episodic memory for edge devices,” in *Proceedings of the First BabyLM Workshop*, (Suzhou, China), pp. 147–154, Association for Computational Linguistics, 2025.
- [7] Hugging Face, “SmolVLM,” 2024.
- [8] X. Chu *et al.*, “MobileVLM: A fast, strong and open vision language assistant for mobile devices,” in *arXiv:2312.16886*, 2023.
- [9] X. Chu *et al.*, “MobileVLM V2: Faster and stronger baseline for vision language model,” in *arXiv:2402.03766*, 2024.
- [10] Y. Zhu *et al.*, “LLaVA-Phi: Efficient multi-modal assistant with small language model,” in *arXiv:2401.02143*, 2024.
- [11] Z. Yuan *et al.*, “TinyGPT-V: Efficient multimodal large language model via small backbones,” 2024. arXiv:2401.07571.
- [12] S. Hu *et al.*, “MiniCPM-V: A gpt-4v level mllm on your phone,” 2024. arXiv:2408.01800.
- [13] H. Wang, S. Ma, L. Dong, *et al.*, “BitNet: Scaling 1-bit transformers for large language models,” in *arXiv:2310.11453*, 2023.
- [14] S. Ma, H. Wang, L. Ma, *et al.*, “The era of 1-bit LLMs: All large language models are in 1.58 bits,” in *arXiv:2402.17764*, 2024.
- [15] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *International Conference on Learning Representations*, 2023.
- [16] J. Lin, J. Tang, H. Tang, *et al.*, “AWQ: Activation-aware weight quantization for llm compression and acceleration,” in *arXiv:2306.00978*, 2023.

- [17] A. Graves, G. Wayne, M. Reynolds, *et al.*, “Hybrid computing using a neural network with dynamic external memory,” in *Nature*, vol. 538, pp. 471–476, 2016.
- [18] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy, “Memorising transformers,” in *International Conference on Learning Representations*, 2022.
- [19] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [20] P. Das *et al.*, “Larimar: Large language models with episodic memory control,” in *arXiv:2403.11901*, 2024.
- [21] A. Rohrbach, L. A. Hendricks, K. Burns, T. Darrell, and K. Saenko, “Object hallucination in image captioning,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2018.
- [22] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J.-R. Wen, “Evaluating object hallucination in large vision-language models,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2023.
- [23] S. Antol, A. Agrawal, J. Lu, M. Mitchell, D. Batra, C. L. Zitnick, and D. Parikh, “VQA: Visual question answering,” in *IEEE International Conference on Computer Vision*, 2015.
- [24] D. A. Hudson and C. D. Manning, “GQA: A new dataset for real-world visual reasoning and compositional question answering,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.
- [25] C.-Y. Hsieh, J. Zhang, Z. Ma, A. Kembhavi, and R. Krishna, “SugarCrepe: Fixing hackable benchmarks for vision-language compositionality,” in *Advances in Neural Information Processing Systems*, 2023.
- [26] J. Choi *et al.*, “Better safe than sorry: A visual emergency recognition dataset for mission-critical deployments,” in *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2026.
- [27] S. Danish *et al.*, “GEO-Bench-VLM: A geospatial disaster understanding benchmark for vision-language models,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
- [28] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in NLP,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.
- [29] D. Patterson *et al.*, “Carbon emissions and large neural network training,” in *arXiv:2104.10350*, 2021.
- [30] K. Lottick *et al.*, “CodeCarbon: Estimate and track carbon emissions from machine learning computing.” <https://codecarbon.io>, 2021.
- [31] M. Oquab *et al.*, “DINOv2: Learning robust visual features without supervision,” in *Transactions on Machine Learning Research*, 2024.

- [32] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [33] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” in *arXiv:1807.03748*, 2018.
- [34] P. Sharma, N. Ding, S. Goodman, and R. Soricut, “Conceptual captions: A cleaned, hypernamed, image alt-text dataset for automatic image captioning,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*, 2018.
- [35] J. Pont-Tuset *et al.*, “Connecting vision and language with localized narratives,” in *European Conference on Computer Vision*, 2020.
- [36] L. Charpentier *et al.*, “BabyLM challenge: Sample-efficient pretraining on a developmentally plausible corpus,” in *BabyLM Workshop, ACL 2025*, 2025.
- [37] L. B. Allal *et al.*, “SmolLM,” 2024.
- [38] L. Yu, P. Poirson, S. Yang, A. C. Berg, and T. L. Berg, “Modeling context in referring expressions,” in *European Conference on Computer Vision*, 2016.
- [39] T.-Y. Lin *et al.*, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision*, 2014.
- [40] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems*, 2023.
- [41] K. Marino, M. Rastegari, A. Farhadi, and R. Mottaghi, “OK-VQA: A visual question answering benchmark requiring external knowledge,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.
- [42] D. Schwenk *et al.*, “A-OKVQA: A benchmark for visual question answering using world knowledge,” in *European Conference on Computer Vision*, 2022.
- [43] H. Altahan *et al.*, “UniBench: Visual reasoning requires rethinking vision-language beyond scaling,” in *arXiv:2408.04810*, 2024.